CS-GY 6923: Lecture 13
PCA and Semantic Embedding, Image Generation

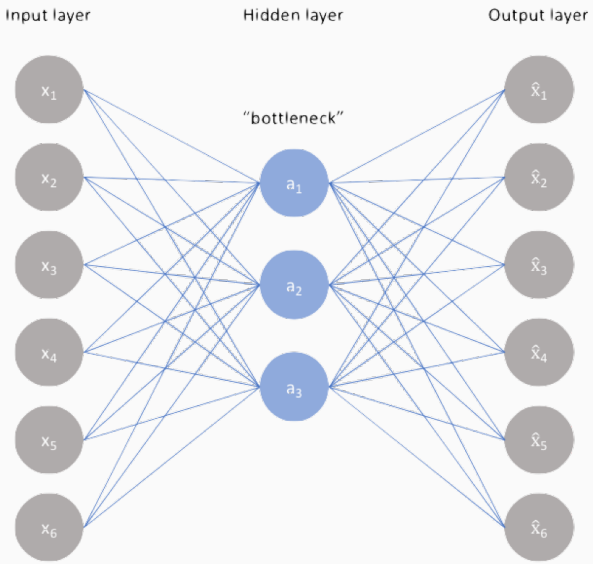
NYU Tandon School of Engineering, Akbar Rafiey

News

- Lab 5 due on Dec 03.
- Homework 5 due on Dec 10.
- Next lecture on Dec 05, the last lecture, will be online over Zoom.

Recap: Principal Component Analysis

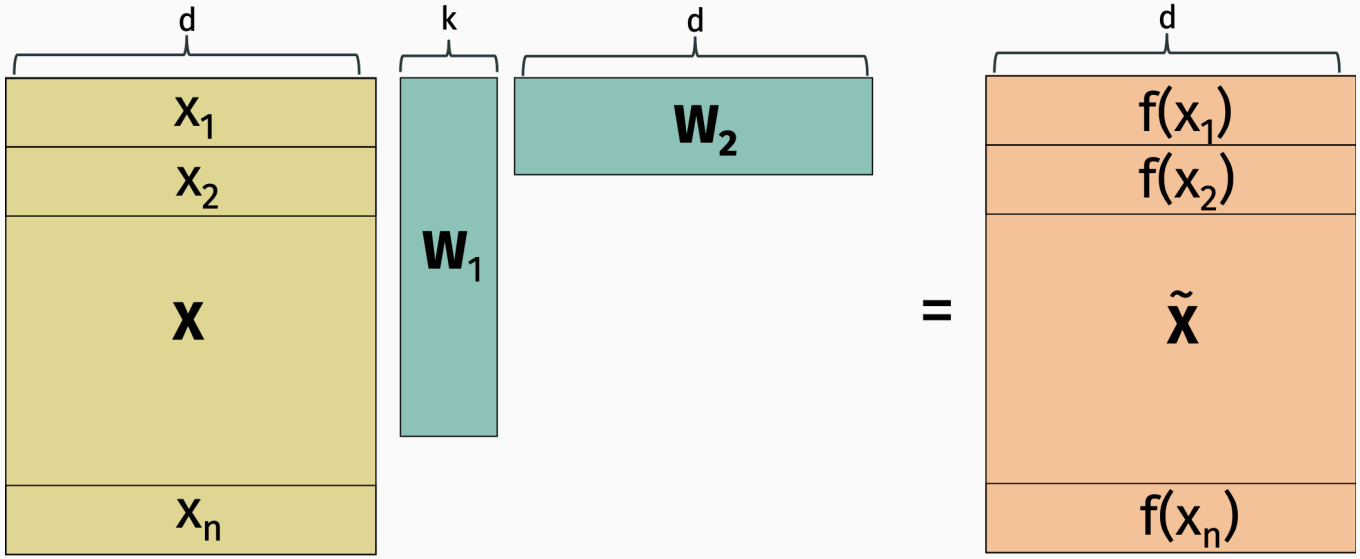
PCA is the "linear regression" of autoencoders:



- Simplest possible model. One layer, no non-linearities.
  - $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{W}_1\mathbf{W}_2$ where $\mathbf{X} \in \mathbb{R}^{n \times d}, \mathbf{W}_1 \in \mathbb{R}^{d \times k}, \mathbf{W}_2 \in \mathbb{R}^{k \times d}$.
  - Want to minimize $\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{X} - \mathbf{X}\mathbf{W}_1\mathbf{W}_2\|_F^2$.

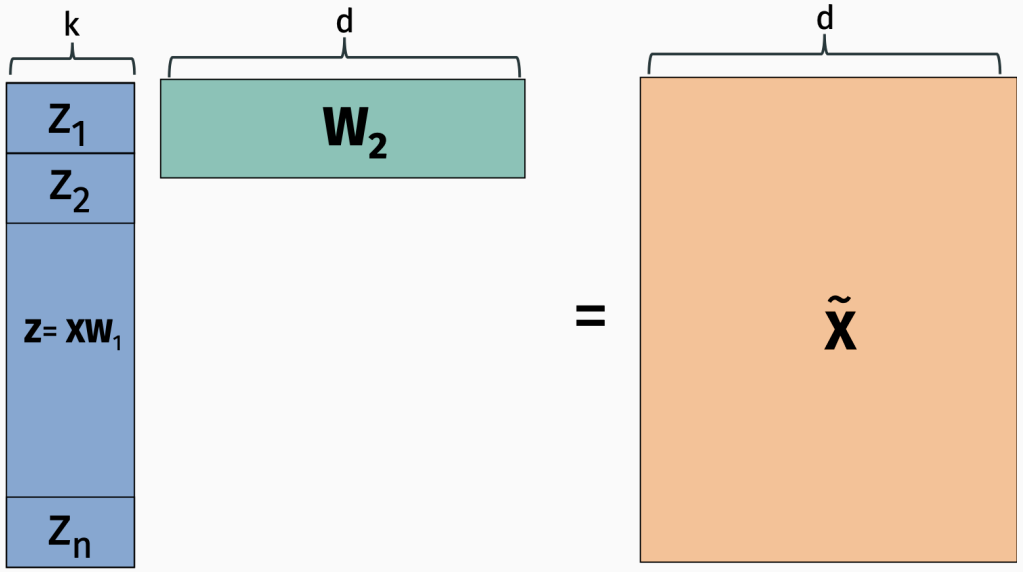
Recap: Principal Component Analysis (PCA)

Given training data set $\mathbf{x}_1, \ldots, \mathbf{x}_n$, let $\mathbf{X}$ denote our data matrix. Let $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{W}_1\mathbf{W}_2$.



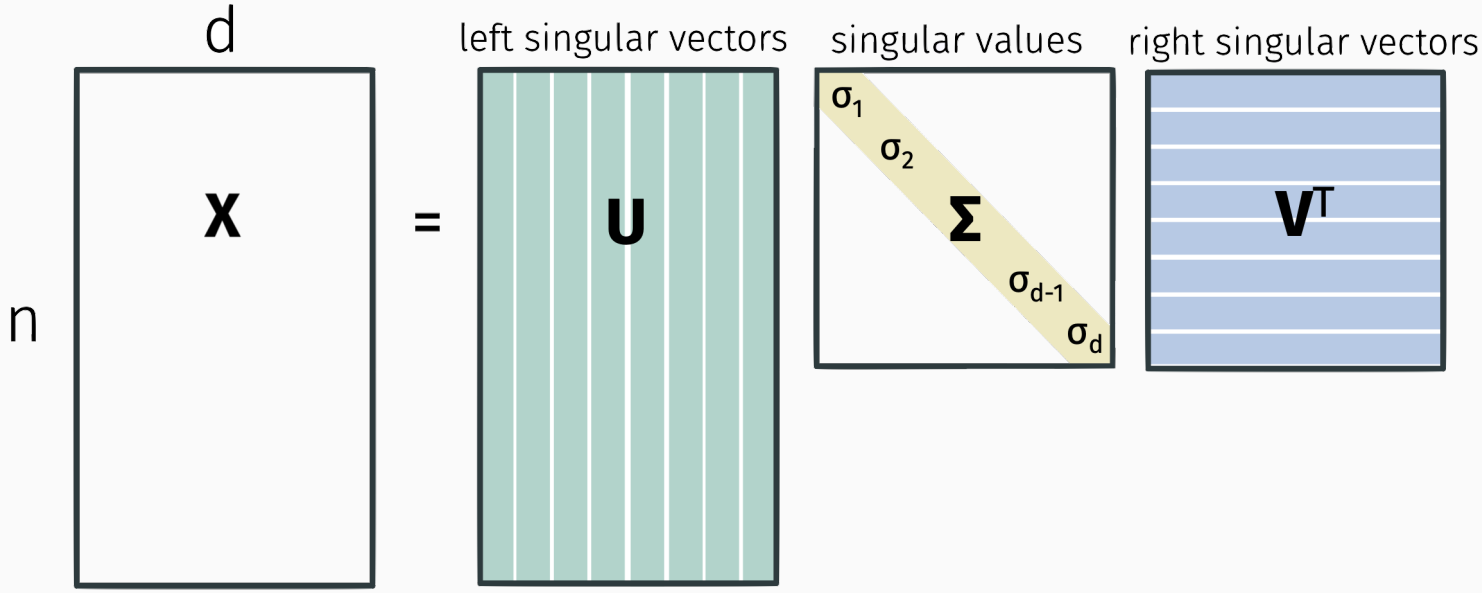
Recap: Low-rank approximation

- $\tilde{\mathbf{X}}$ is a **low-rank matrix**. It only has rank (at most) k for $k \ll d$.
- Finding best $\mathbf{W}_1, \mathbf{W}_2$: equivalent to low-rank approximation. Can be efficiently and provable optimized using the SVD.



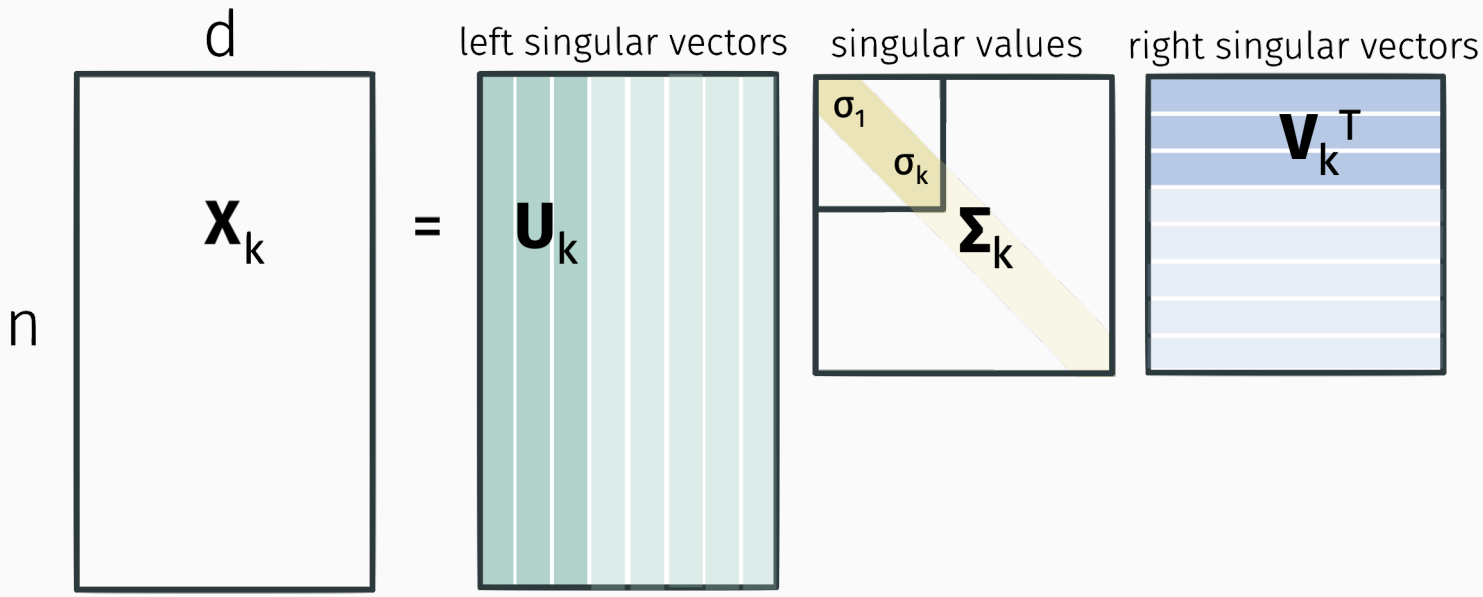
Recap: Singular Value Decomposition

Any matrix $\mathbf{X}$ can be written:



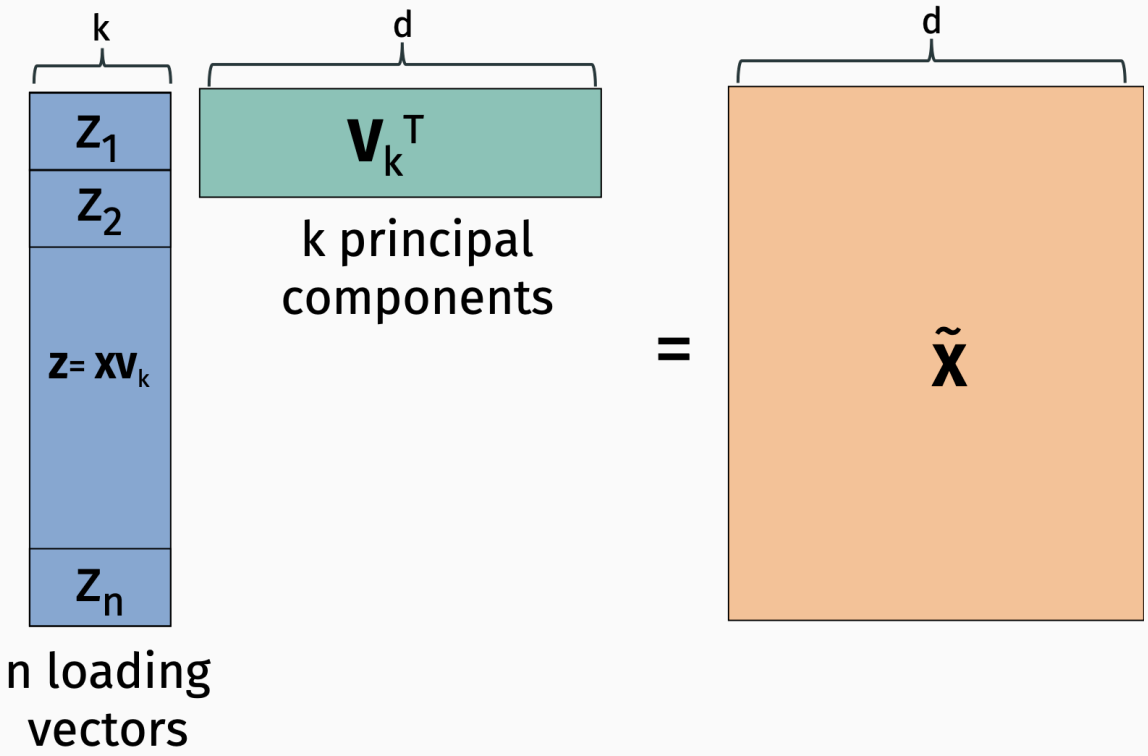
Where $\mathbf{U}^T\mathbf{U} = \mathbf{I}$, $\mathbf{V}^T\mathbf{V} = \mathbf{I}$, and $\sigma_1 \geq \sigma_2 \geq \ldots \sigma_d \geq 0$. I.e. $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices. Can be computed in $O(nd^2)$ time (faster with approximation algos).

Recap: Partial Singular Value Decomposition



Can be computed in roughly $O(ndk)$ time.

Recap: Principal Component Analysis

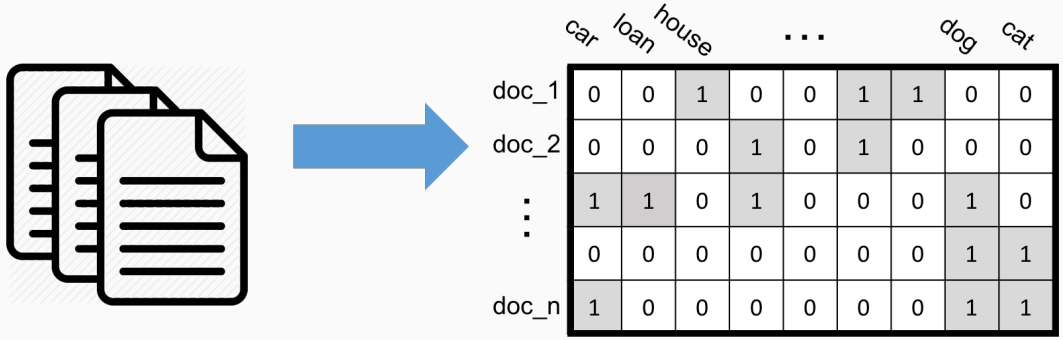


Eckart–Young–Mirsky Theorem: $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^T$ is the optimal low-rank approximation to $\mathbf{X}$. So $\mathbf{W}_1 = \mathbf{V}_k$ and $\mathbf{W}_2 = \mathbf{V}_k^T$ are optimal autoencoder parameters.

PCA for Term Document

Term document matrix

Word-document matrices tend to be low rank.

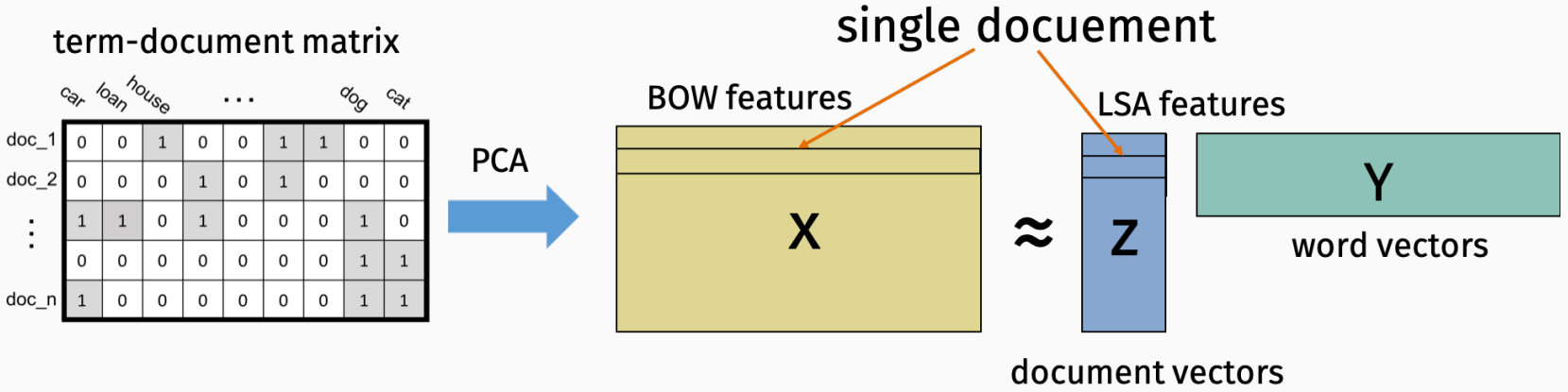


Documents tend to fall into a relatively small number of different categories, which use similar sets of words:

- **Financial news:** *markets, analysts, dow, rates, stocks*
- **US Politics:** *president, senate, pass, slams, twitter, media*
- **StackOverflow posts:** *python, help, convert, javascript*

Latent semantic analysis

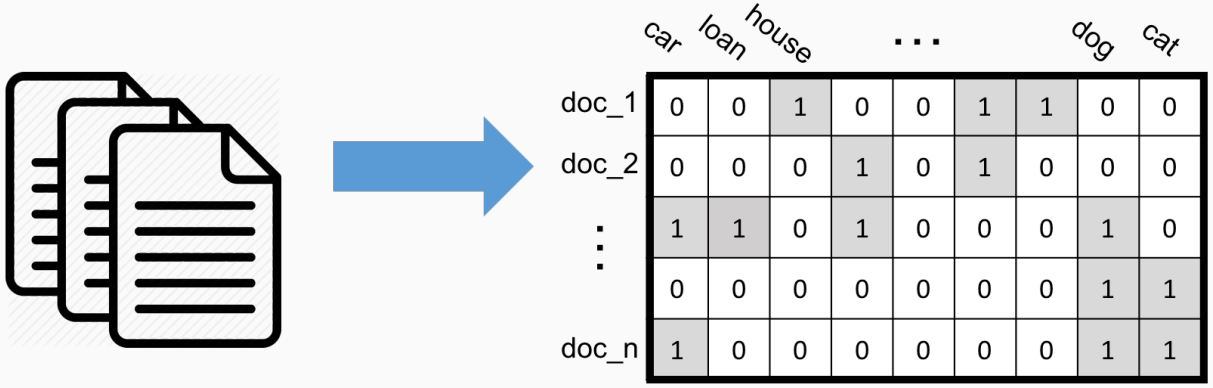
Latent semantic analysis = PCA applied to a word-document matrix (usually from a large corpus). One of the most fundamental techniques in **natural language processing** (NLP).

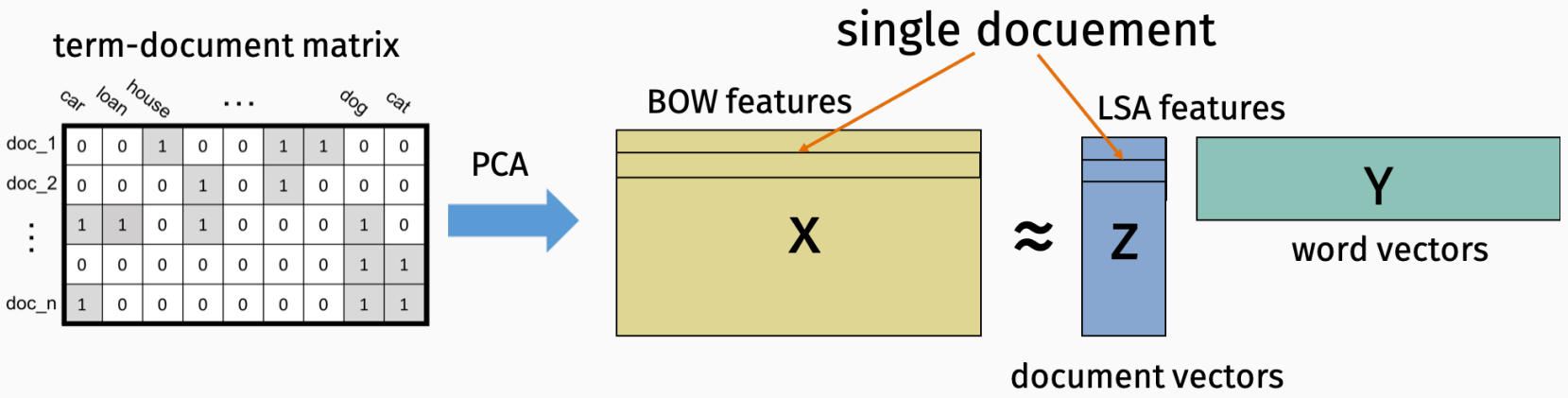


Each column of **z** corresponds to a latent "category" or "topic". Corresponding row in **Y** corresponds to the "frequency" with which different words appear in documents on that topic.

Latent Semantic Analysis (LSA)

Word-document matrix:





For documents with a lot of shared words, $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ is a large positive number.

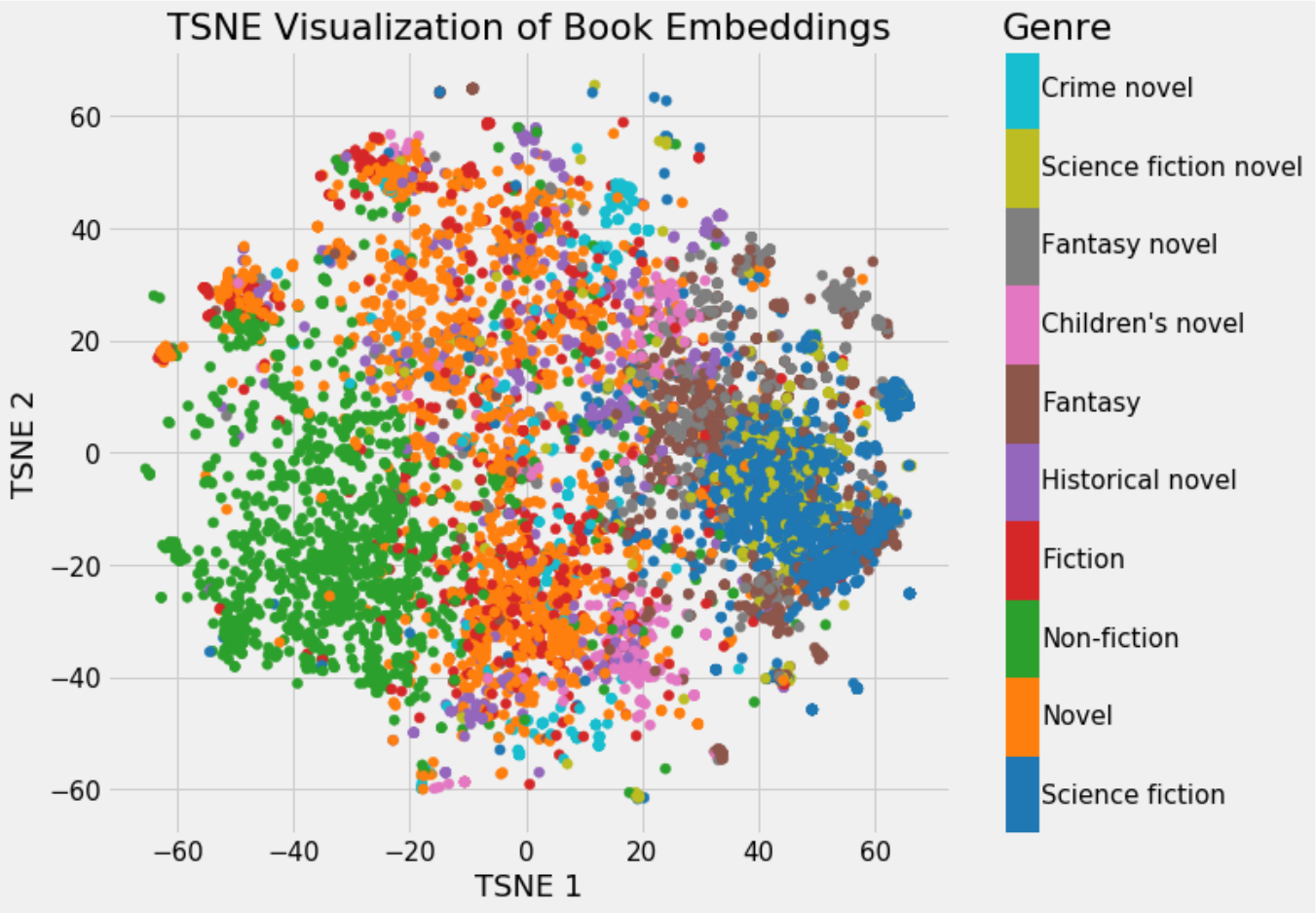
Latent semantic analysis

Similar documents have similar LSA document vectors. I.e. $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$ is large.

- $\mathbf{z}_i$ provides a more compact "finger print" for documents than the long bag-of-words vectors. Useful for e.g search engines.
- Comparing document vectors is often more effective than comparing raw BOW features. Two documents can have $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$ large even if they have no overlap in words. E.g. because both share a lot of words with words with another document k, or with a bunch of other documents.

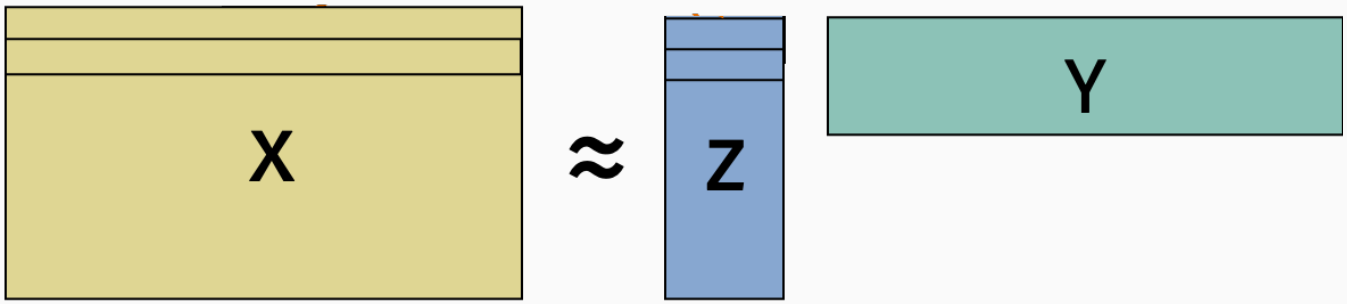
Document embeddings

For similar documents, $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$ should be large. I.e. $\mathbf{z}_i$ and $\mathbf{z}_j$ point in the same direction.

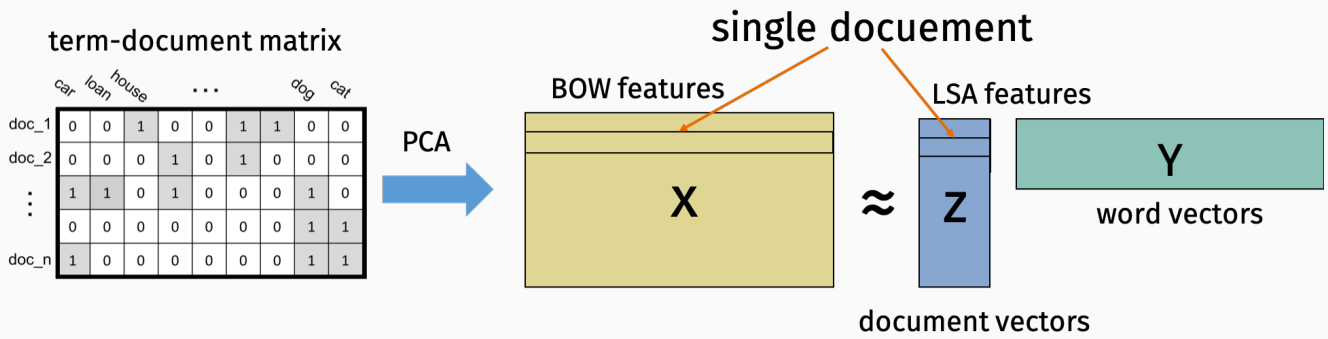


From PCA to semantic embeddings

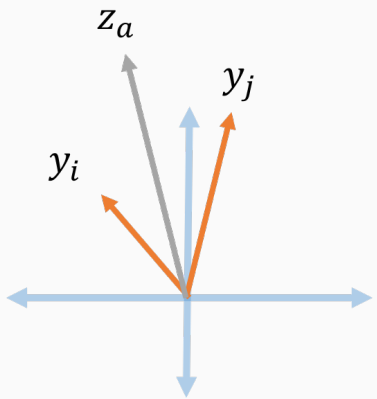
Simple but useful observation: The i, j entry of $\tilde{\mathbf{X}}$ equals $\langle \mathbf{z}_i, \mathbf{y}_j \rangle$.



Word embeddings



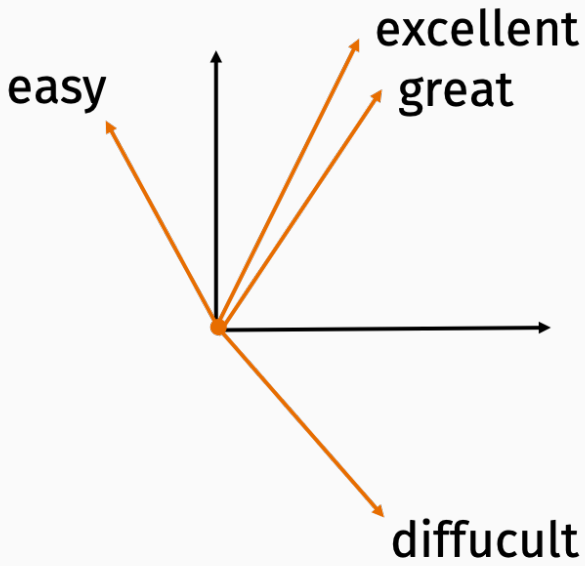
- $\langle \mathbf{y}_i, \mathbf{z}_a \rangle \approx 1$ when doc_a contains $word_i$.
- If $word_i$ and $word_j$ both appear in doc_a, then $\langle \mathbf{y}_i, \mathbf{z}_a \rangle \approx \langle \mathbf{y}_j, \mathbf{z}_a \rangle \approx 1$, so we expect $\langle \mathbf{y}_j, \mathbf{y}_j \rangle$ to be large.



If two words appear in the same document their, word vectors tend to point more in the same direction.

Word embeddings

Result: Map words to numerical vectors in a semantically meaningful way. Similar words map to similar vectors. Dissimilar words to dissimilar vectors.

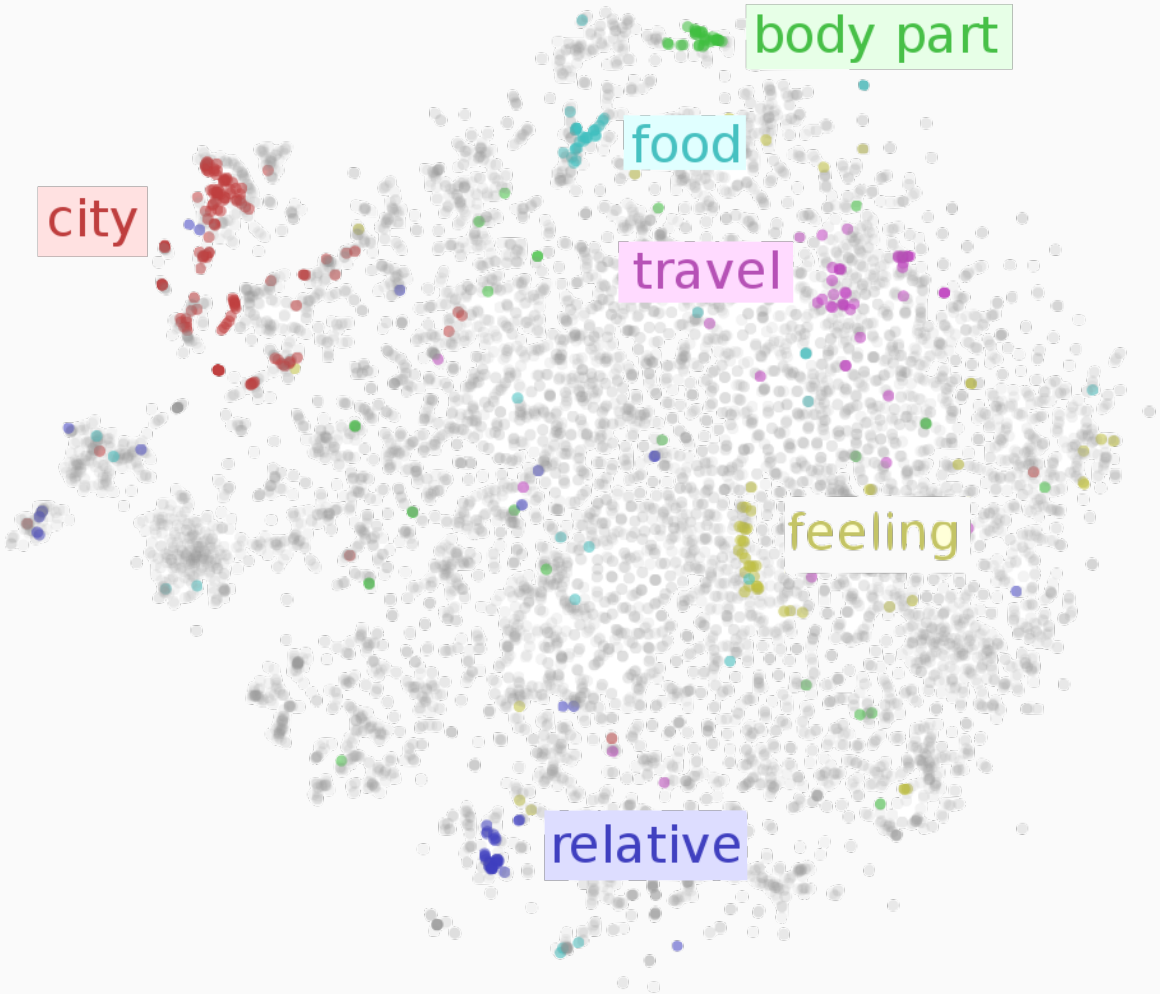


Extremely useful "side-effect" of LSA.

Capture e.g. the fact that "great" and "excellent" are near synonyms. Or that "difficult" and "easy" are antonyms.

Word embeddings

For similar words, $\langle \mathbf{y}_i, \mathbf{y}_j \rangle$ should be large. I.e. $\mathbf{y}_i$ and $\mathbf{y}_j$ point in the same direction.



Word embeddings: motivating problem

Review 1: *Very small and handy for traveling or camping. Excellent quality, operation, and appearance.*

Review 2: *So far this thing is great. Well designed, compact, and easy to use. I'll never use another can opener.*

Review 3: *Not entirely sure this was worth* $20. *Mom couldn't figure out how to use it and it's fairly difficult to turn for someone with arthritis.*

Goal is to classify reviews as "positive" or "negative".

Bag-of-words features

Vocabulary: Small, handy, excellent, great, quality, compact, easy, difficult.

Review 1: *Very small and handy for traveling or camping. Excellent quality, operation, and appearance.*

[, , , , , , ,]

Review 2: *So far this thing is great. Well designed, compact, and easy to use. I'll never use another can opener.*

[, , , , , , ,]

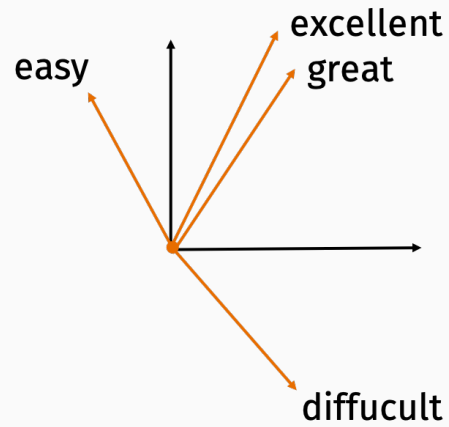
Review 3: *Not entirely sure this was worth* $20*. Mom couldn't figure out how to use it and it's fairly difficult to turn for someone with arthritis.*

[, , , , , , ,]

Semantic embeddings

Bag-of-words approach typically only works for large data sets.

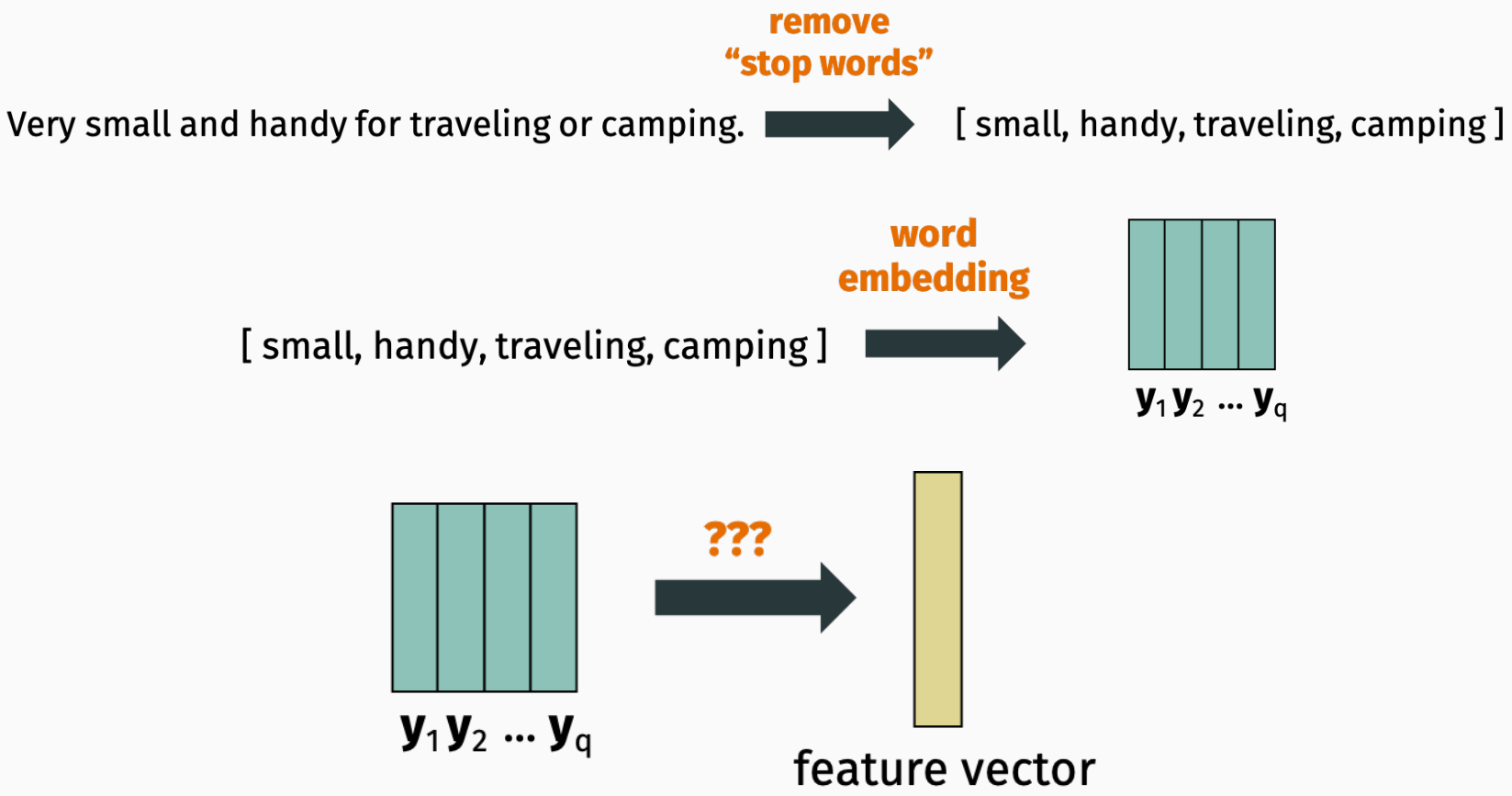
The features do not capture the fact that "great" and "excellent" are near synonyms. Or that "difficult" and "easy" are antonyms.



This can be addressed by first mapping words to semantically meaningful vectors. That mapping can be trained using a much large corpus of text than the data set you are working with (e.g. Wikipedia, Twitter, news data sets).

Using word embeddings

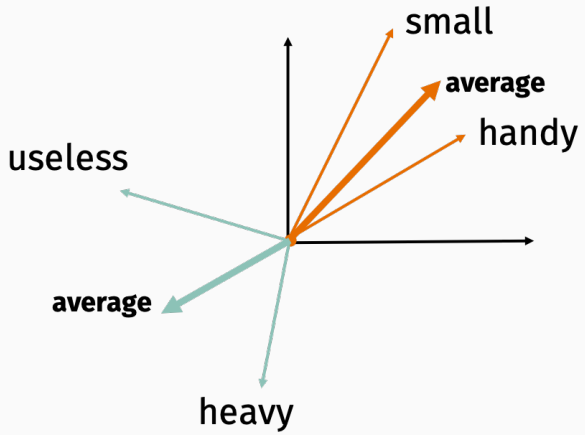
How to go from word embeddings to features for a whole sentence or chunk of text?



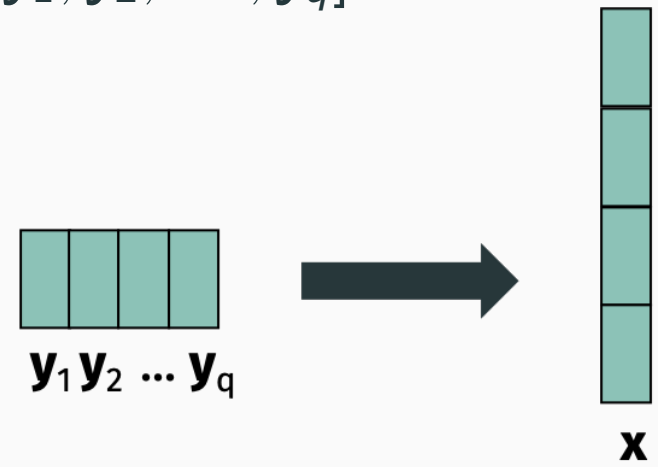
Using word embeddings

A few simple options:

Feature vector $\mathbf{x} = \frac{1}{q} \sum_{i=1}^{q} \mathbf{y}_q$.

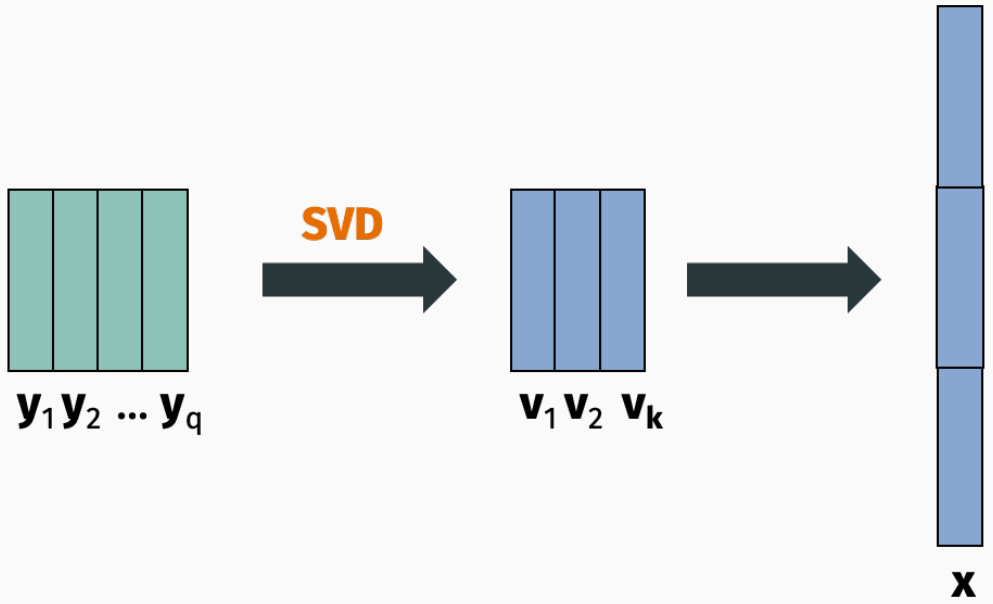


Feature vector $\mathbf{x} = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_q]$.



Using word embeddings

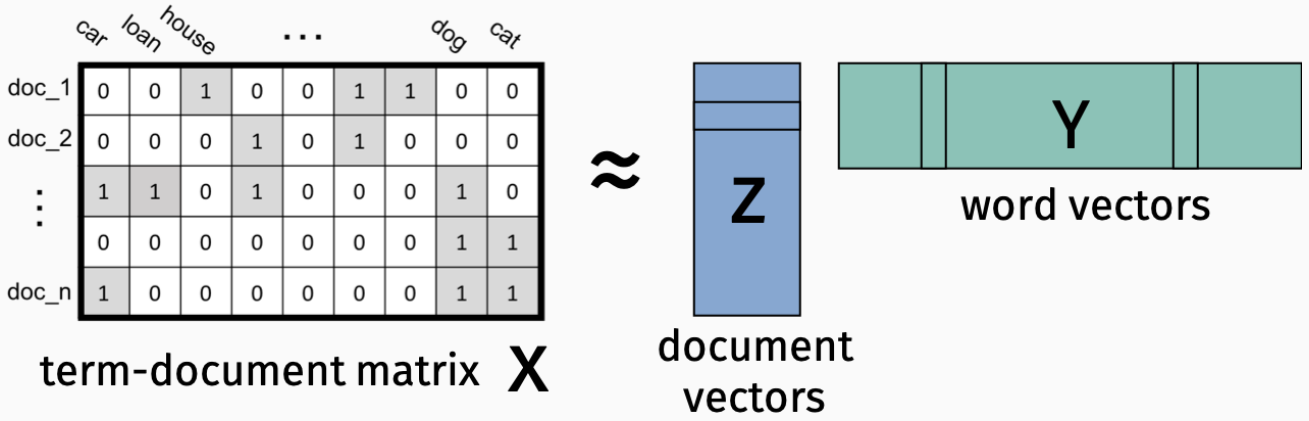
To avoid issues with inconsistent sentence length, word ordering, etc., can concatenate a fixed number of top principal components of the matrix of word vectors:



There are much more complicated approaches that account for word position in a sentence. Lots of pretrained libraries available (e.g. Facebook's `InferSent`).

Word embeddings

Another view on word embeddings from LSA:

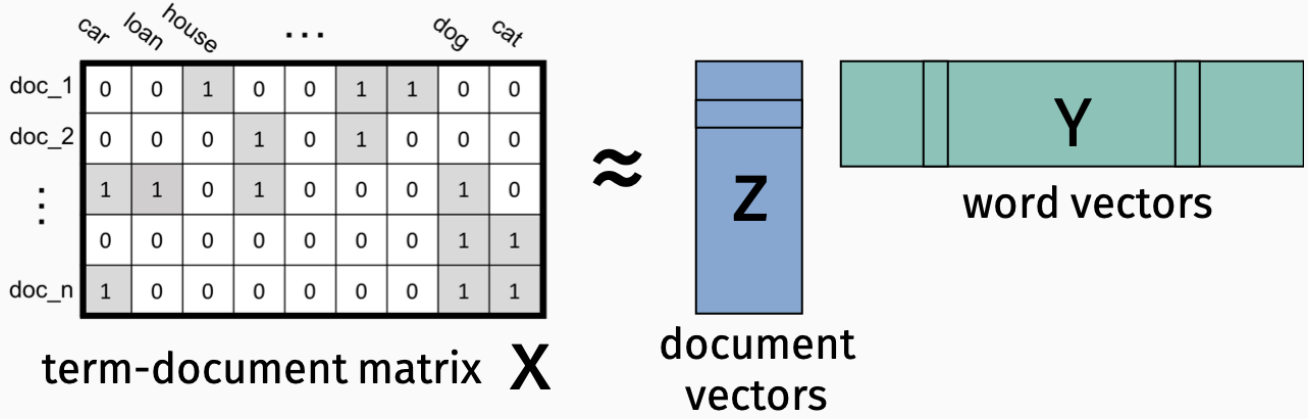


We chose $\mathbf{Z}$ to equal $\mathbf{X}\mathbf{V}_k = \mathbf{U}_k\mathbf{\Sigma}_k$ and $\mathbf{Y} = \mathbf{V}_k^T$.

Could have just as easily set $\mathbf{Z} = \mathbf{U}_k$ and $\mathbf{Y} = \mathbf{\Sigma}_k\mathbf{V}_k^T$, so $\mathbf{Z}$ has orthonormal columns.

Word embeddings

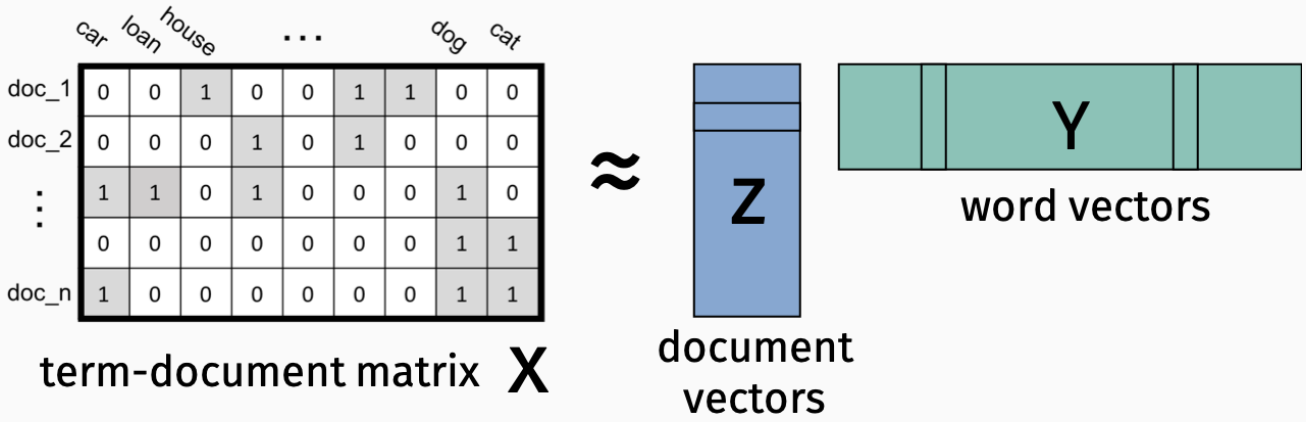
Another view on word embeddings from LSA:



- $\mathbf{X} \approx \mathbf{ZY}$
- $\mathbf{X}^T\mathbf{X} \approx \mathbf{Y}^T\mathbf{Z}^T\mathbf{ZY} = \mathbf{Y}^T\mathbf{Y}$
- So for $word_i$ and $word_j$, $\langle \mathbf{y}_i, \mathbf{y}_j \rangle \approx [\mathbf{X}^T\mathbf{X}]_{i,j}$.

What does the i, j entry of $\mathbf{X}^T\mathbf{X}$ reprent?

Word embeddings



What does the i, j entry of $\mathbf{X}^T\mathbf{X}$ represent?

The number of documents where words i and j were both used.

Word embeddings

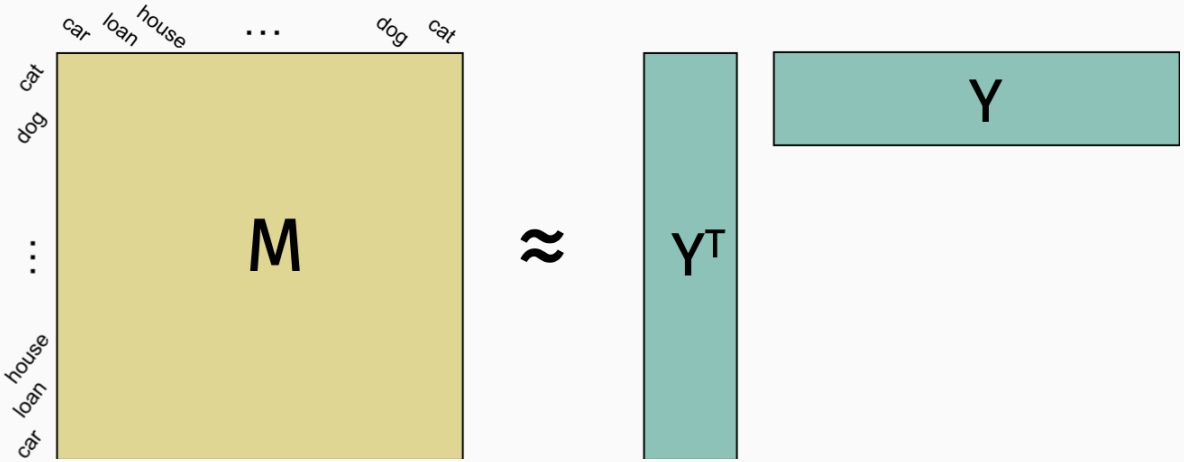
$\langle \mathbf{y}_i, \mathbf{y}_j \rangle$ is larger if $word_i$ and $word_j$ appear in more documents together (high value in **word-word co-occurrence matrix**, $\mathbf{X}^T\mathbf{X}$). Similarity of word embeddings mirrors similarity of word context.

General word embedding recipe:

1. Choose similarity metric $k(word_i, word_j)$ which can be computed for any pair of words.
2. Construct similarity matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ with $\mathbf{M}_{i,j} = k(word_i, word_j)$.
3. Find low rank approximation $\mathbf{M} \approx \mathbf{Y}^T\mathbf{Y}$ where $\mathbf{Y} \in \mathbb{R}^{k \times n}$.
4. Columns of $\mathbf{Y}$ are word embedding vectors.

We expect that $\langle \mathbf{y}_i, \mathbf{y}_j \rangle$ will be larger for more similar words.

Word embeddings



How do current state-of-the-art methods differ from LSA?

- Similarity based on co-occurrence in smaller chunks of words. E.g. in sentences or in any consecutive sequences of 3, 4, or 10 words.
- Usually transformed in non-linear way. E.g. $k(word_i, word_j) = \frac{p(i,j)}{p(i)p(j)}$ where $p(i,j)$ is the frequency both i, j appeared together, and $p(i)$, $p(j)$ is the frequency either one appeared.

Modern word embeddings

Computing word similarities for "window size" 4:

The girl walks to her dog to the park.
It can take a long time to park your car in NYC.
The dog park is always crowded on Saturdays.

The girl walks to her dog to the park.
It can take a long time to park your car in NYC.
The dog park is always crowded on Saturdays.

The girl walks to her dog to the park.
It can take a long time to park your car in NYC.
The dog park is always crowded on Saturdays.

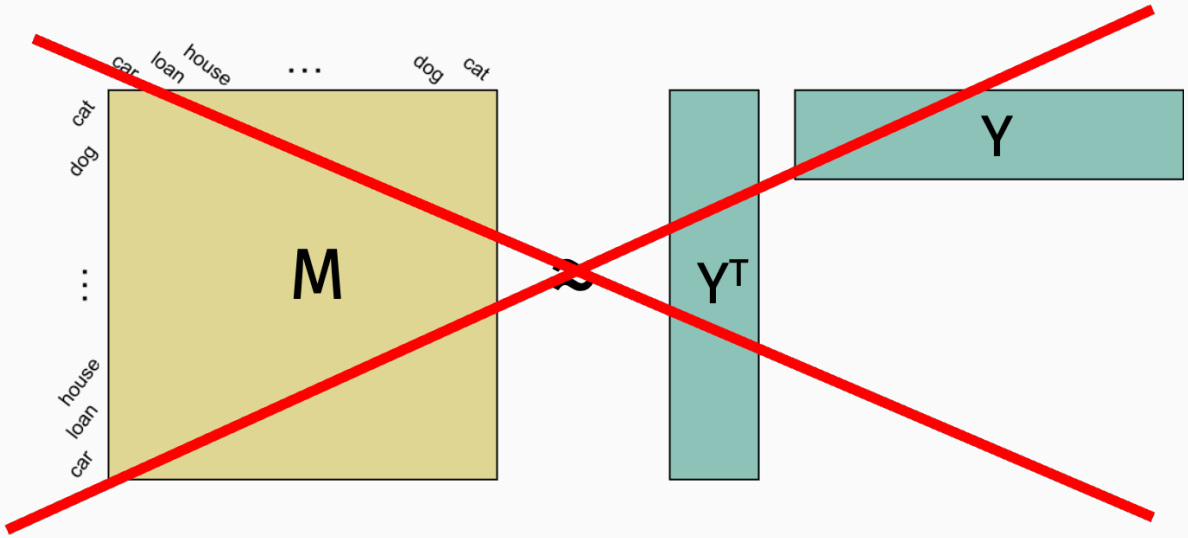
	dog	park	crowded	the
dog	0	2	0	3
park	2	0	1	2
crowded	0	1	0	0
the	3	2	0	0

Modern word embeddings

Current state of the art models: `GloVE`, `word2vec`.

- `word2vec` was originally presented as a shallow neural network model, but it is equivalent to matrix factorization method (Levy, Goldberg 2014).
- For `word2vec`, similarity metric is the "point-wise mutual information": $\log \frac{p(i,j)}{p(i)p(j)}$.

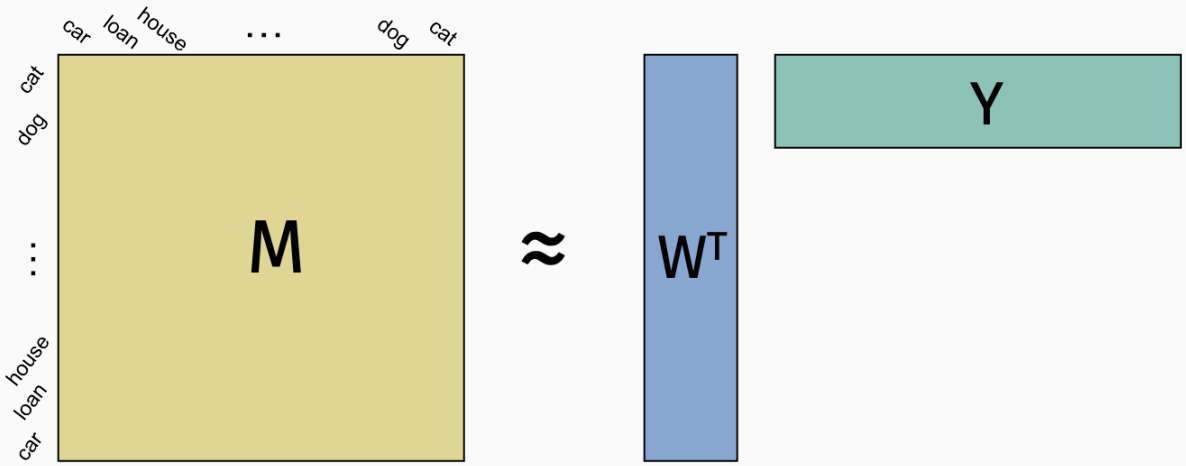
Caveat about factorization



SVD will not return a symmetric factorization in general. In fact, if $\mathbf{M}$ is not positive semidefinite[1] then the optimal low-rank approximation does not have this form.

[1] I.e., $k(word_i, word_j)$ is not a positive semidefinite kernel.

Caveat about factorization



- For each word i we get a left and right embedding vector $\mathbf{w}_i$ and $\mathbf{y}_i$. It's reasonable to just use one or the other.
- If $\langle \mathbf{y}_i, \mathbf{y}_j \rangle$ is large and positive, we expect that $\mathbf{y}_i$ and $\mathbf{y}_j$ have similar similarity scores with other words, so they typically are still related words.
- Another option is to use as your features for a word the concatenation $[\mathbf{w}_i, \mathbf{y}_i]$

Easiest way to use word embeddings

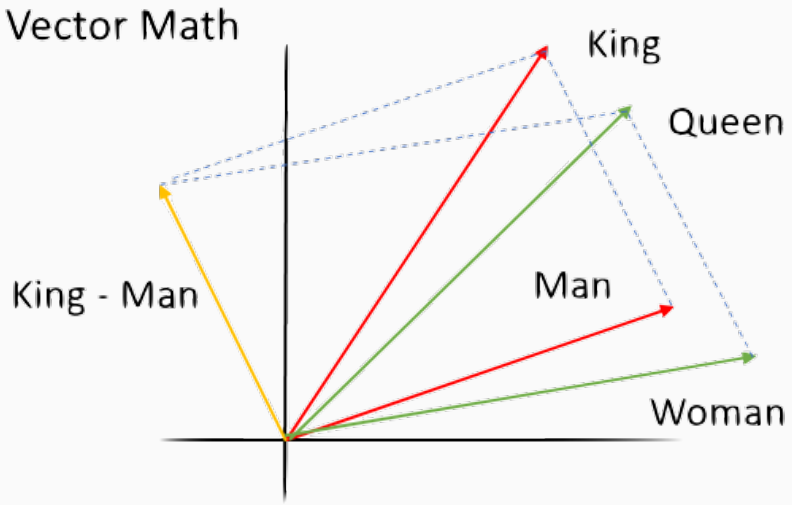
Lots of pre-trained word vectors are available online:

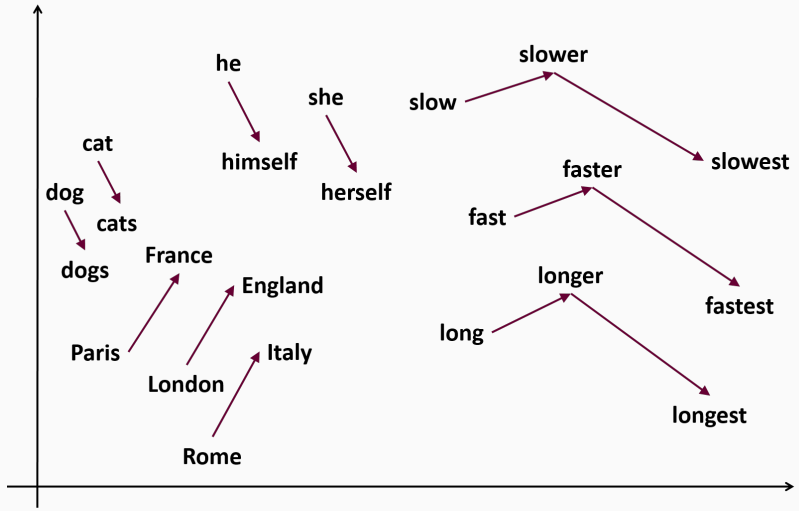
- **Original gloVe website:**
  `https://nlp.stanford.edu/projects/glove/`.
- **Compilation of many sources:**
  `https://github.com/3Top/word2vec-api`

Word embeddings math

Lots of cool demos for what can be done with these embeddings.

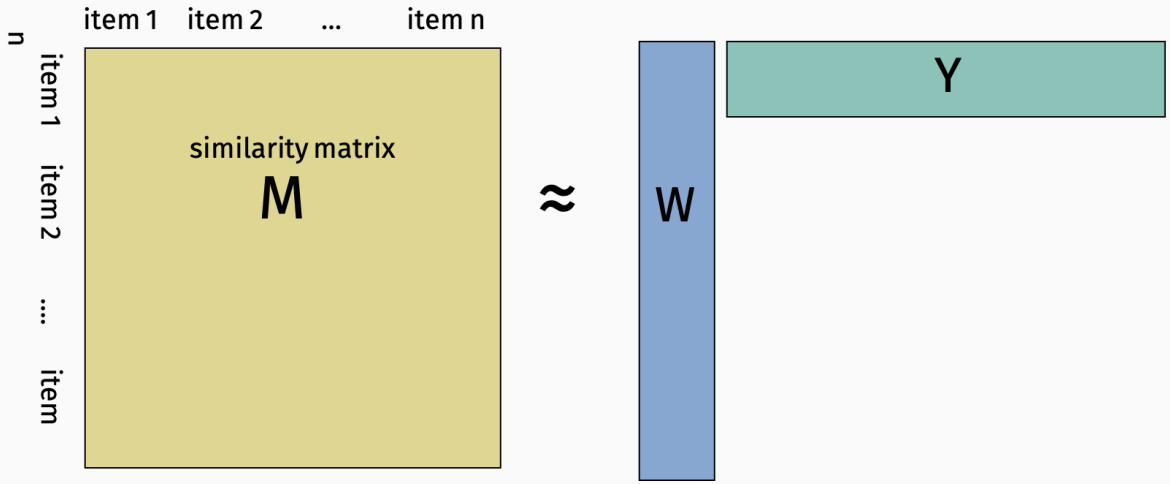
E.g. "vector math" to solve analogies.





Semantic embeddings for other data

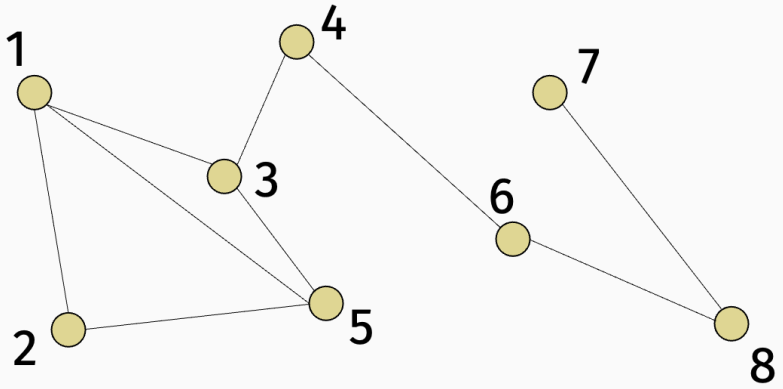
The same approach used for word embeddings can be used to obtain meaningful numerical features for any other data where there is a natural notion of similarity.



For example, the items could be nodes in a social network graph. Maybe be want to predict an individuals age, level of interest in a particular topic, political leaning, etc.

Node embeddings

Generate random walks (e.g. "sentences" of nodes) and measure similarity by node co-occurence frequency.

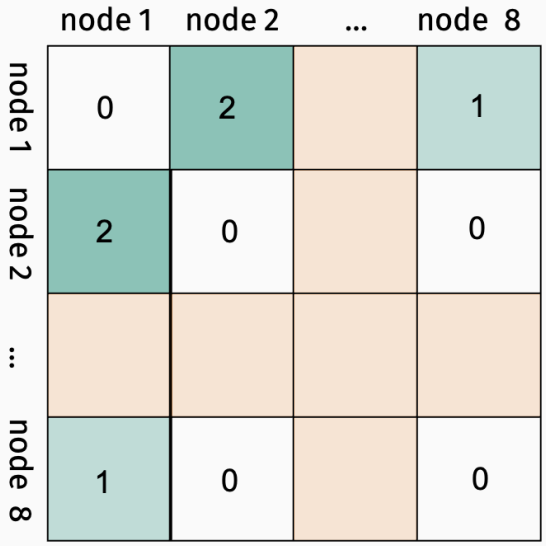


1, 3, 4, 4, 5, 2, 1, 2, 5
6, 8, 6, 4, 3, 1, 5, 3, 4
7, 8, 6, 8, 7, 8, 6, 8, 6
⋮
4, 6, 8, 6, 4, 3, 1, 2, 5

Node embeddings

Again typically normalized and apply a non-linearity (e.g. log) as in word embeddings.

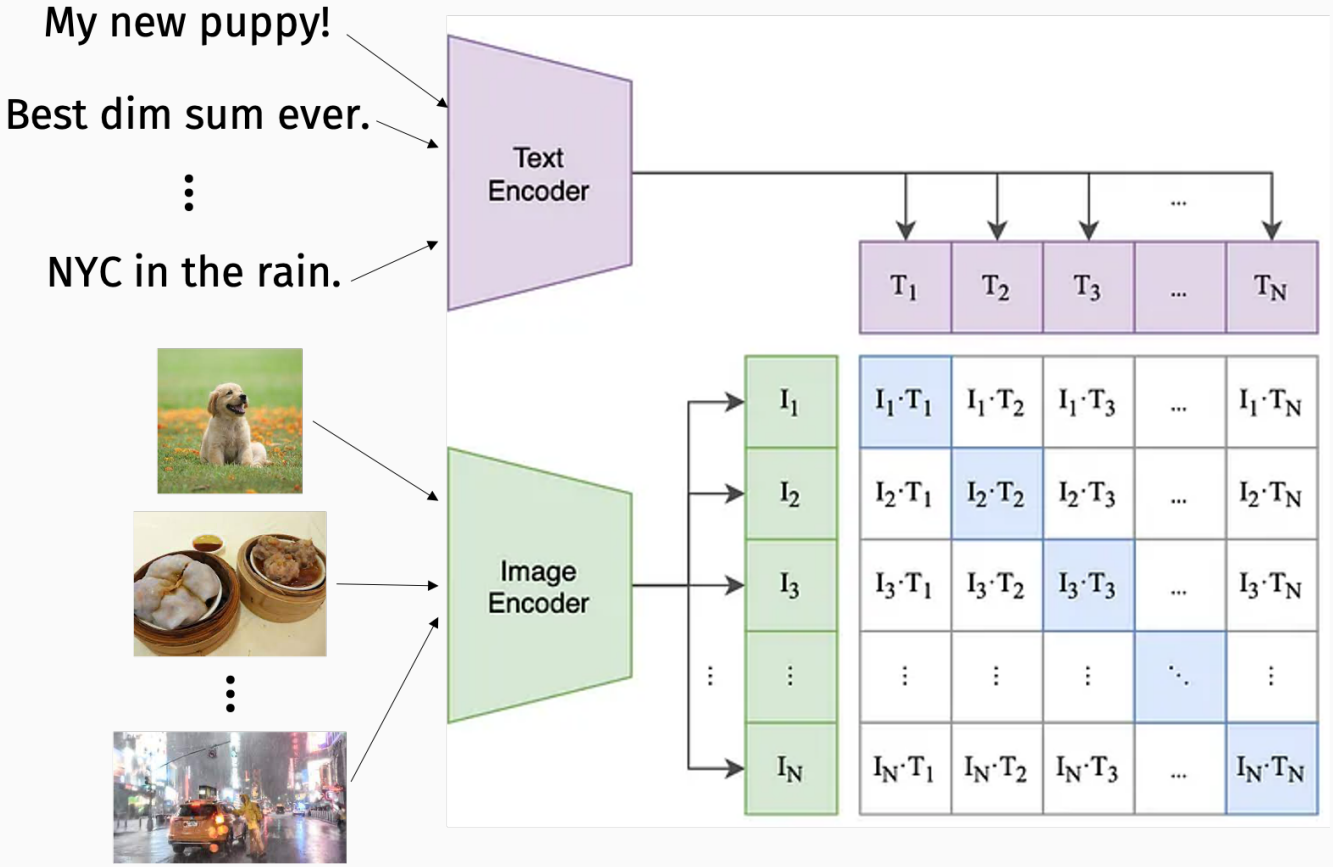
1, 3, 4, 4, 5, 2, 1, 2, 5
6, 8, 6, 4, 3, 1, 5, 3, 4
7, 8, 6, 8, 7, 8, 6, 8, 6
⋮
4, 6, 8, 6, 4, 3, 1, 2, 5



Popular implementations: `DeepWalk`, `Node2Vec`. Again initially derived as simple neural network models, but are equivalent to matrix-factorization (Qiu et al. 2018).

Bimodal embeddings: text and image

We can also create embeddings that represent different types of data. OpenAI's clip architecture:



Goal: Train embedding architectures so that $\langle \mathbf{T}_i, \mathbf{I}_j \rangle$ are similar if image and sentence are similar.

Clip training

What do we use as ground truth similarities during training? Sample a batch of sentence/image pairs and just use identity matrix.



My new puppy!	1	0	0
Best dim sum ever.	0	1	0
NYC in the rain.	0	0	1

This is called contrastive learning. Train unmatched text/image pairs to have nearly orthogonal embedding vectors.

Clip for zero-shot learning

Learning Transferable Visual Models From Natural Language Supervision

Alec Radford [*1] Jong Wook Kim [*1] Chris Hallacy [1] Aditya Ramesh [1] Gabriel Goh [1] Sandhini Agarwal [1]
Girish Sastry [1] Amanda Askell [1] Pamela Mishkin [1] Jack Clark [1] Gretchen Krueger [1] Ilya Sutskever [1]

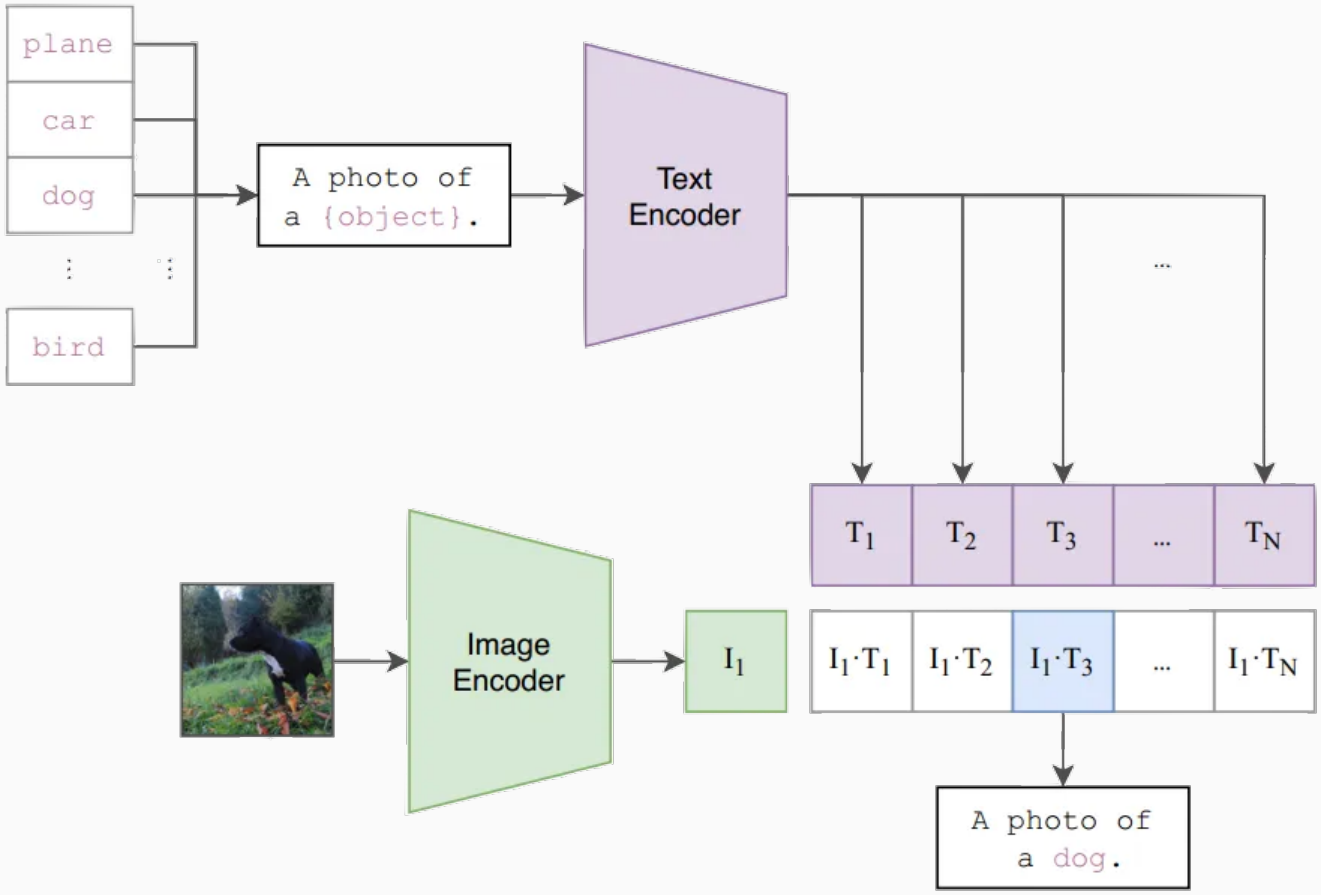
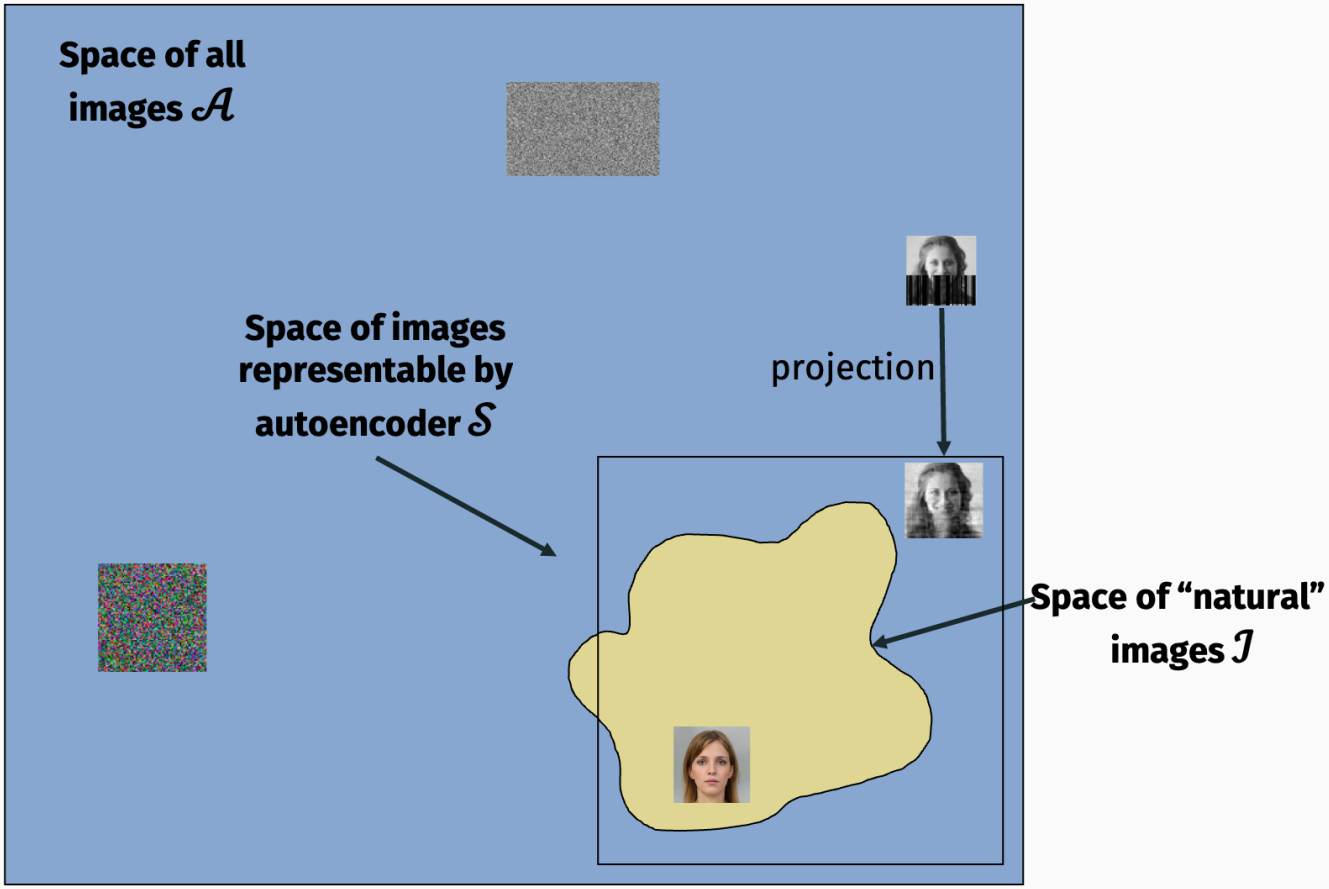


Image Synthesis

Recap: Autoencoders learn compressed representations

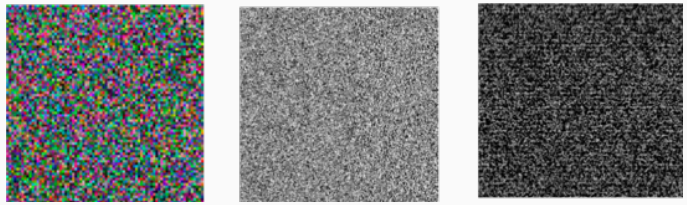


$f(\mathbf{x}) = d(e(\mathbf{x}))$ projects an image $\mathbf{x}$ closer to the space of natural images.

Autoencoders for data generation

Suppose we want to generate a random natural image. How might we do that?

- **Option 1**: Draw each pixel value in $\mathbf{x}$ uniformly at random. Draws a random image from $\mathcal{A}$.

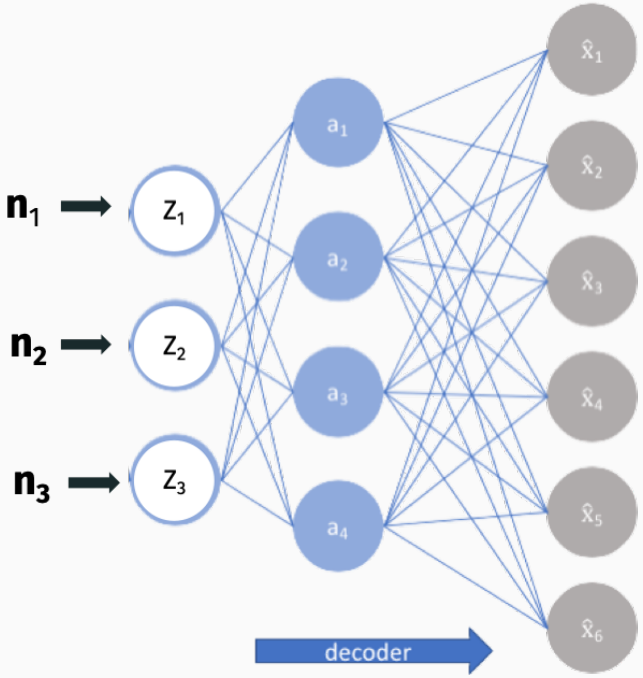


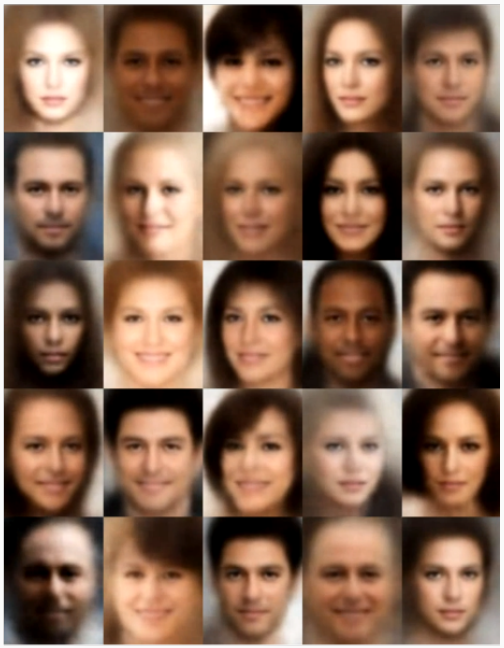
- **Option 2**: Draw $\mathbf{x}$ randomly from $\mathcal{S}$, the space of images representable by the autoencoder.

How do we randomly select an image from $\mathcal{S}$?

Autoencoders for data generation

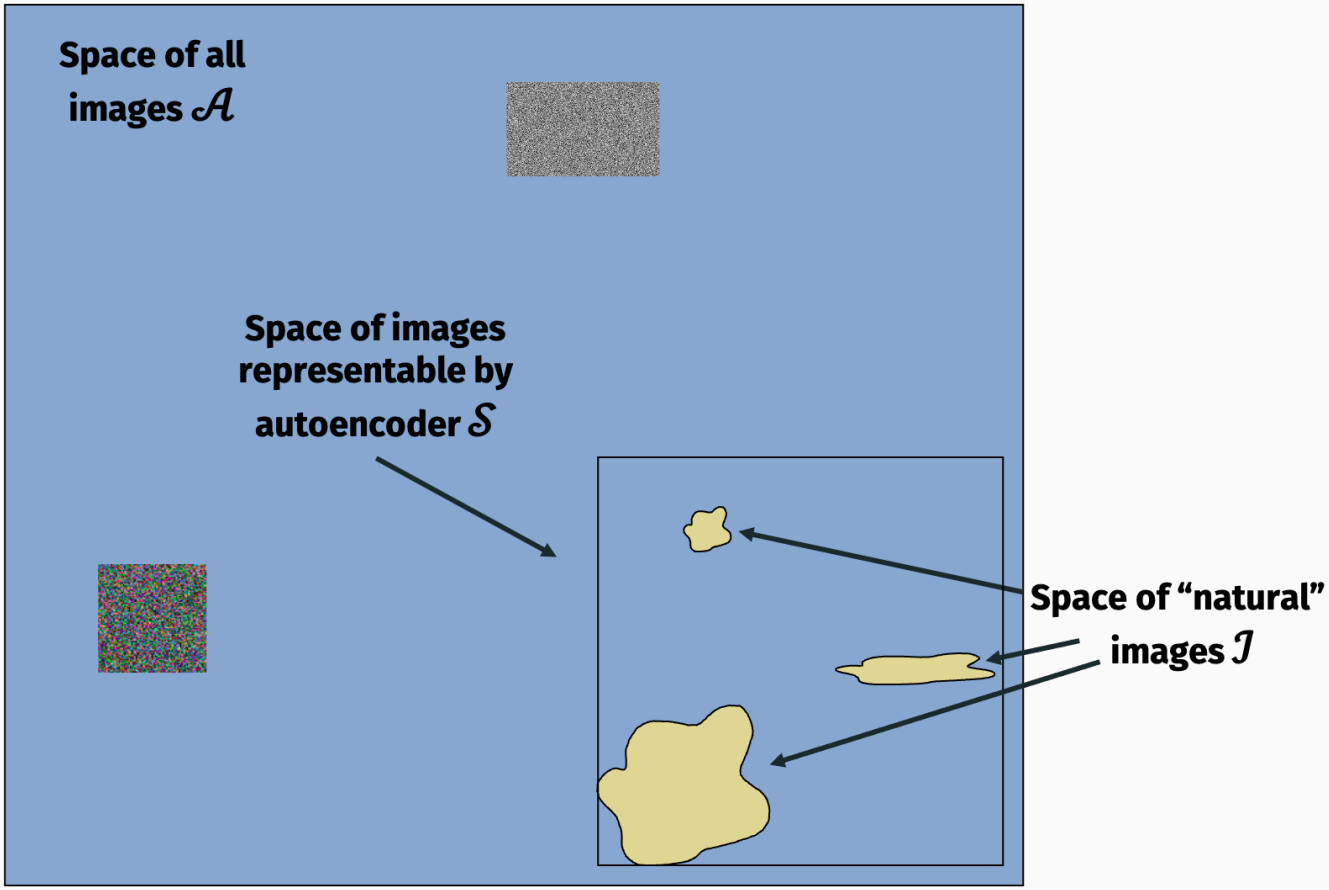
Autoencoder approach to generative ML: Feed random inputs into decoder to produce random realistic outputs.





Main issue: most random inputs will "miss" and produce garbage results.

Autoencoders for data generation



Variational Auto-Encoders (VAEs) attempt to resolve this issue.

Variational AutoEncoders (VAEs)

VAEs attempt to resolve this issue. Basic ideas:

- Instead of mapping inputs to a single latent vector, VAEs map them to a probability distribution in the latent space (e.g., a Gaussian distribution)

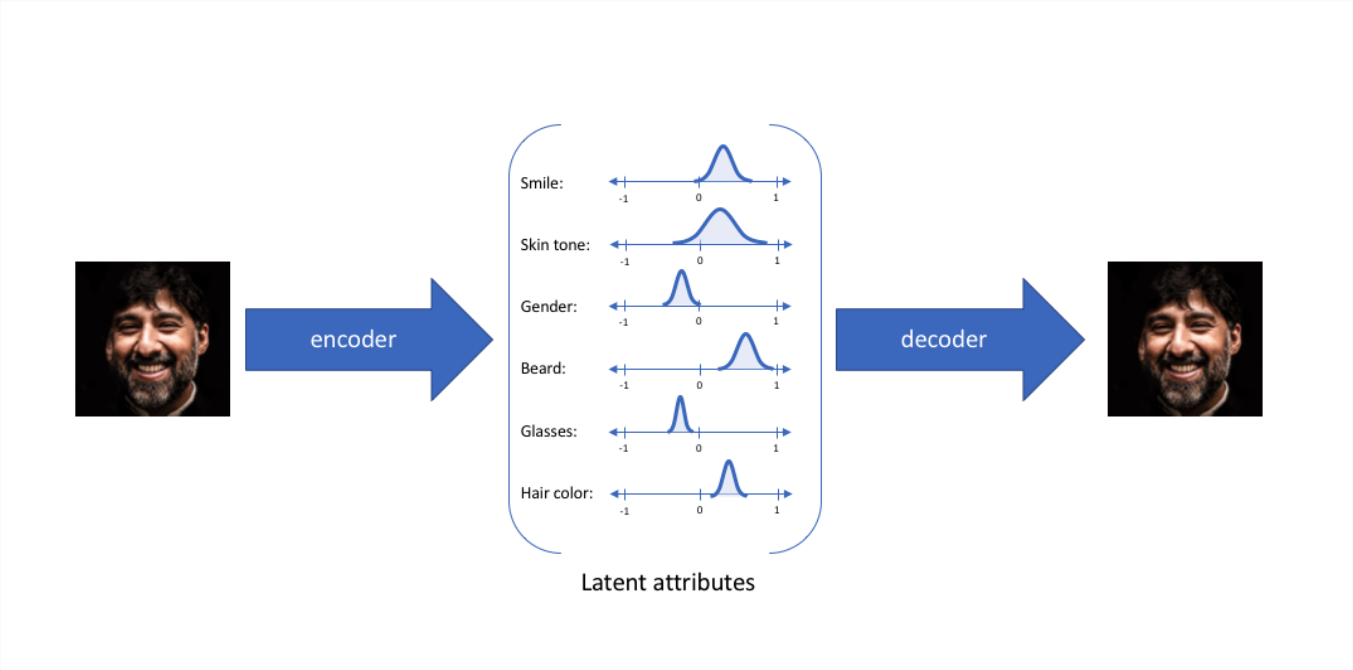


Image from `https://www.jeremyjordan.me/variational-autoencoders/`

Variational AutoEncoders (VAEs)

Basic ideas:

- Suppose there exists some hidden variable $\boldsymbol{z}$ which generates $\boldsymbol{x}$.
- Ideally we want to understand $p(\boldsymbol{z} \mid \boldsymbol{x})$ (probabilistic encoder) and $p(\boldsymbol{x} \mid \boldsymbol{z})$ (probabilistic decoder)
- We can only see the data. Computing $p(\boldsymbol{z} \mid \boldsymbol{x})$ is hard

$$p(\boldsymbol{z} \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid \boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{x})}$$

Variational AutoEncoders (VAEs)

Basic ideas:

- Let's approximate $p(\boldsymbol{z} \mid \boldsymbol{x})$ using a simpler to understand distribution $q(\boldsymbol{z} \mid \boldsymbol{x})$.

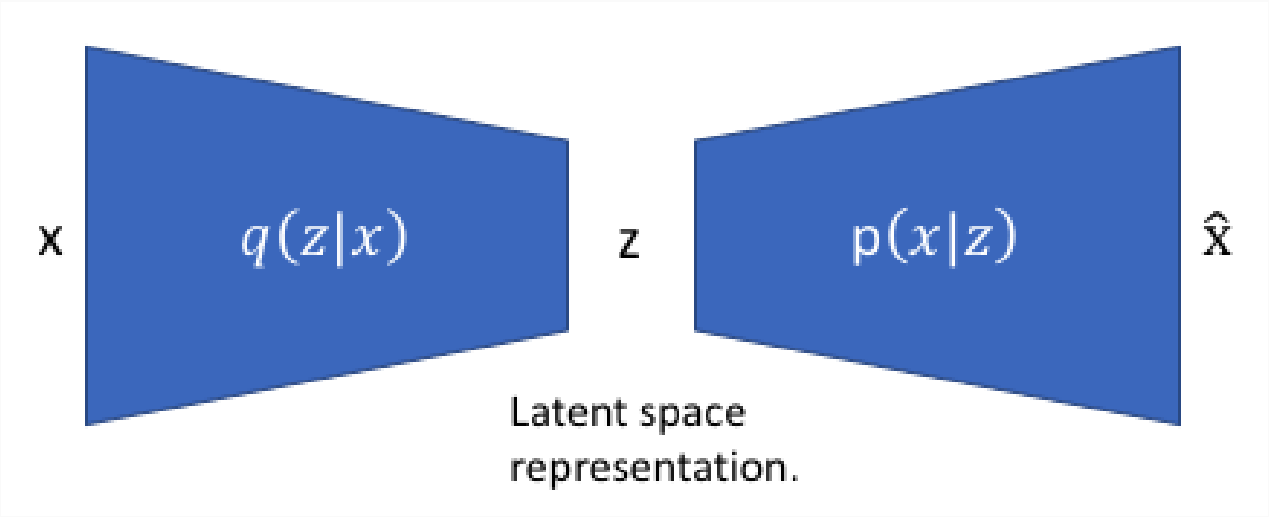


Image from https://www.jeremyjordan.me/variational-autoencoders/

Variational AutoEncoders (VAEs)

Basic ideas:

- Let's approximate $p(\boldsymbol{z} \mid \boldsymbol{x})$ using a simpler to understand distribution $q(\boldsymbol{z} \mid \boldsymbol{x})$.
- $q(\boldsymbol{z} \mid \boldsymbol{x})$ must have nice properties e.g., it should be as similar as possible to $p(\boldsymbol{z} \mid \boldsymbol{x})$
- Optimization problem !

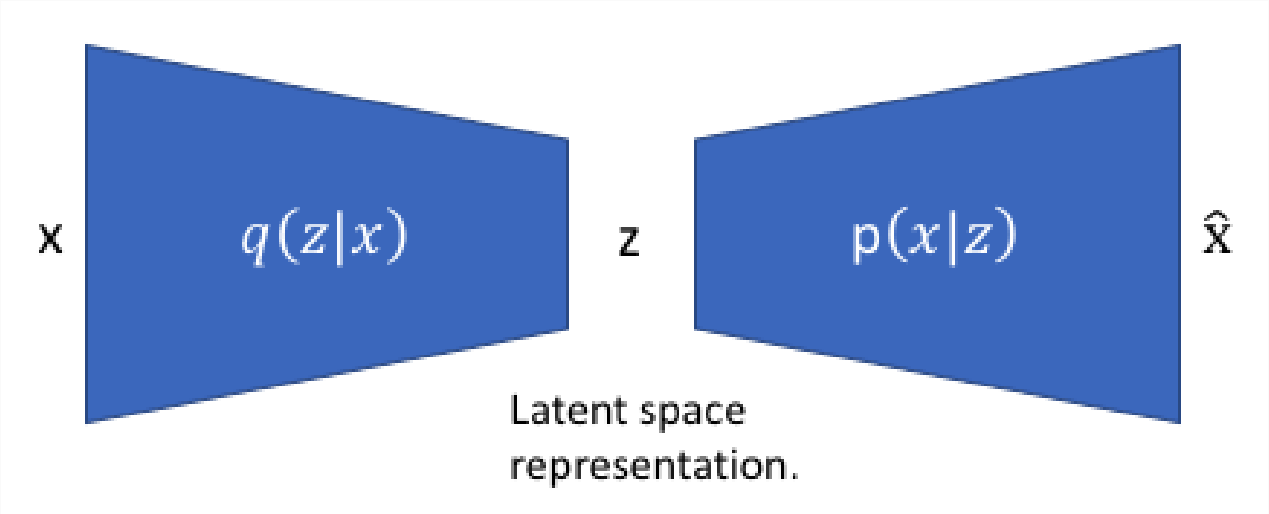


Image from https://www.jeremyjordan.me/variational-autoencoders/

Kullback–Leibler divergence

KL divergence is a measure of difference between two probability distributions.

$$KL(P, Q) = \sum_x P(x) \frac{\log(P(x))}{\log Q(x)}$$

Back to our optimization problem: $q(\boldsymbol{z} \mid \boldsymbol{x})$ and $p(\boldsymbol{z} \mid \boldsymbol{x})$ should be similar

$$\min KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z} \mid \boldsymbol{x}))$$

Equivalent to:

$$\max \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x})} \log(p(\boldsymbol{x} \mid \boldsymbol{z})) - KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z}))$$

VAE objective

Back to our optimization problem: $q(\boldsymbol{z} \mid \boldsymbol{x})$ and $p(\boldsymbol{z} \mid \boldsymbol{x})$ should be similar

$$\min KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z} \mid \boldsymbol{x}))$$

Equivalent to:

$$\max \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x})} \log(p(\boldsymbol{x} \mid \boldsymbol{z})) - KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z}))$$

What is the first term? what is the second term?

VAE objective

Back to our optimization problem: $q(\boldsymbol{z} \mid \boldsymbol{x})$ and $p(\boldsymbol{z} \mid \boldsymbol{x})$ should be similar

$$\min KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z} \mid \boldsymbol{x}))$$

Equivalent to:

$$\max \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x})} \log(p(\boldsymbol{x} \mid \boldsymbol{z})) - KL(q(\boldsymbol{z} \mid \boldsymbol{x}), p(\boldsymbol{z}))$$

First term: reconstruction likelihood.

Second term: ensures that our learned distribution q is similar to the true prior distribution p.

VAEs: implementation

Have neural networks to learn the mappings $q(\boldsymbol{z} \mid \boldsymbol{x})$ and $p(\boldsymbol{x} \mid \boldsymbol{z})$.

$$\max_{\theta,\phi} \mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})} \log(p_\theta(\boldsymbol{x} \mid \boldsymbol{z})) - KL(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}), p_\theta(\boldsymbol{z}))$$

Assumptions: $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$, $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$, and $p_\theta(\boldsymbol{z})$ are Gaussian distributions.

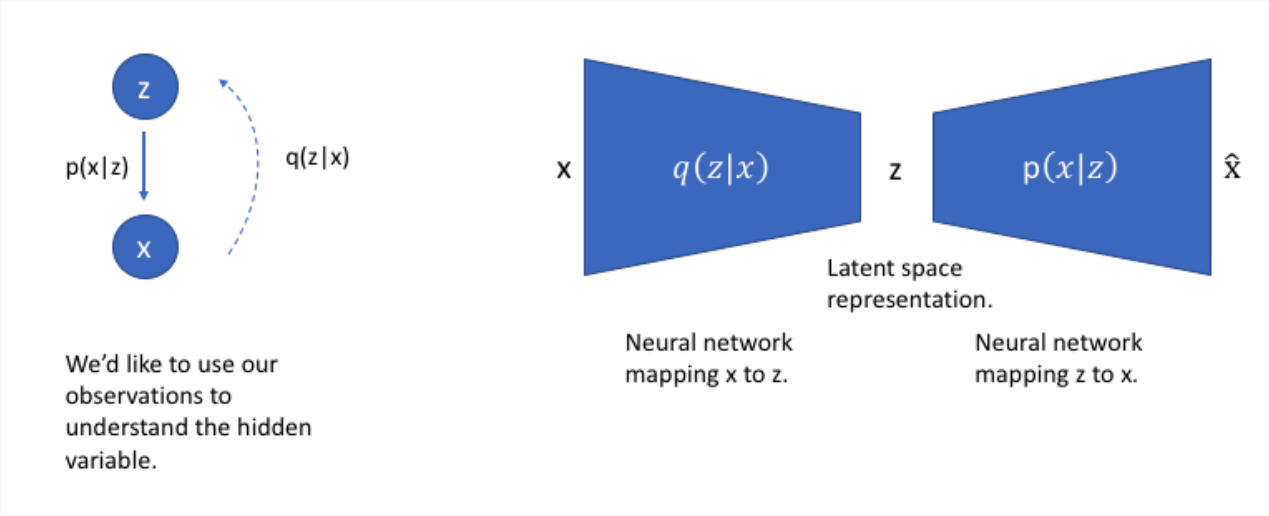


Image from `https://www.jeremyjordan.me/variational-autoencoders/`

VAEs: implementation

The encoder model of a VAE will output parameters describing a distribution for each dimension in the latent space.

Assuming Gaussian we only need a mean and a variance for describing each dimension in the latent space

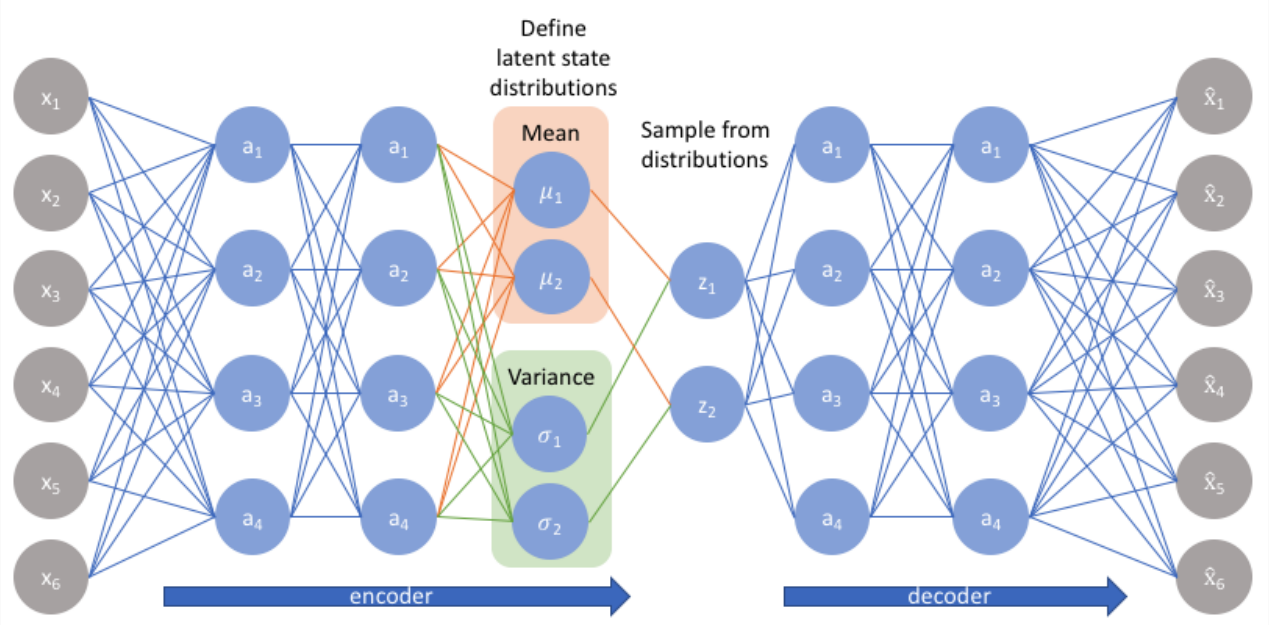


Image from `https://www.jeremyjordan.me/variational-autoencoders/`

VAEs: implementation

It is not easy to backpropagate the gradient through samples !

Reparameterization technique addresses this issue.

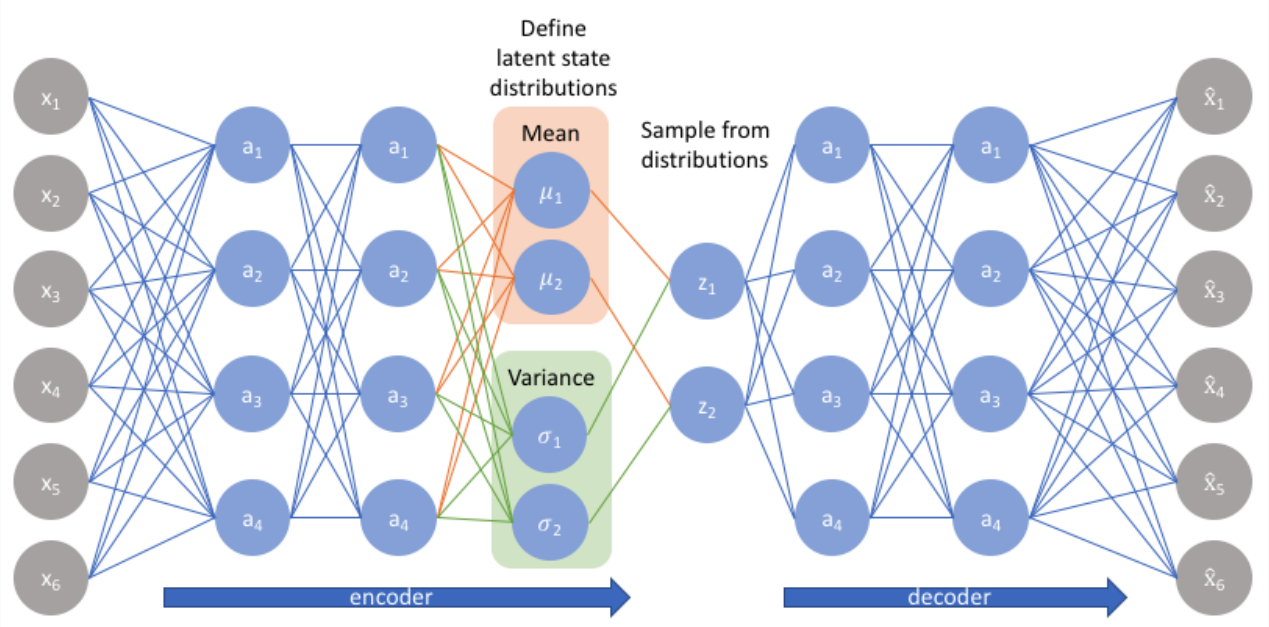


Image from `https://www.jeremyjordan.me/variational-autoencoders/`

VAEs: reparameterization technique

$\boldsymbol{z} \sim q_\phi(\cdot \mid \boldsymbol{x})$ is normally distributed, as

$$\mathcal{N}(\mu_\phi(\boldsymbol{x}), \sigma_\phi(\boldsymbol{x}))$$

This can be reparameterized by letting $\varepsilon \sim \mathcal{N}(0, I)$, and constructing $\boldsymbol{z}$ as

$$\boldsymbol{z} = \mu_\phi(\boldsymbol{x}) + L_\phi(\boldsymbol{x})\varepsilon$$

Here, $\sigma_\phi(\boldsymbol{x})$ is obtained by the Cholesky decomposition:

$$\sigma_\phi(\boldsymbol{x}) = L_\phi(\boldsymbol{x}) L_\phi(\boldsymbol{x})^T$$

Then we have

$$\nabla_\phi \mathbb{E}_{\boldsymbol{z} \sim q_\phi(\cdot|\boldsymbol{x})} \left[\ln \frac{p_\theta(\boldsymbol{x}, \boldsymbol{z})}{q_\phi(\boldsymbol{z}|\boldsymbol{x})} \right] = \mathbb{E}_\varepsilon \left[\nabla_\phi \ln \frac{p_\theta(\boldsymbol{x}, \mu_\phi(\boldsymbol{x}) + L_\phi(\boldsymbol{x})\varepsilon)}{q_\phi(\mu_\phi(\boldsymbol{x}) + L_\phi(\boldsymbol{x})\varepsilon|\boldsymbol{x})} \right]$$

and so we obtain an unbiased estimator of the gradient, allowing stochastic gradient descent.

VAEs: reparameterization technique

Since we reparameterized z, we need to find $q_\phi(\boldsymbol{z}|\boldsymbol{x})$. Let q_0 be the probability density function for ε, then

$$\ln q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \ln q_0(\varepsilon) - \ln \left|\det \left(J(\boldsymbol{z}, \varepsilon)\right)\right|$$

Since $\boldsymbol{z} = \mu_\phi(\boldsymbol{x}) + L_\phi(\boldsymbol{x})\varepsilon$, this becomes

$$\ln q_\phi(\boldsymbol{z}|\boldsymbol{x}) = -\frac{1}{2}\|\varepsilon\|^2 - \ln |\det L_\phi(\boldsymbol{x})| - \frac{n}{2}\ln(2\pi)$$

VAEs: reparameterization technique

We can now optimize the parameters of the distribution (unbiased estimation of the gradient) while still maintaining the ability to randomly sample from that distribution.

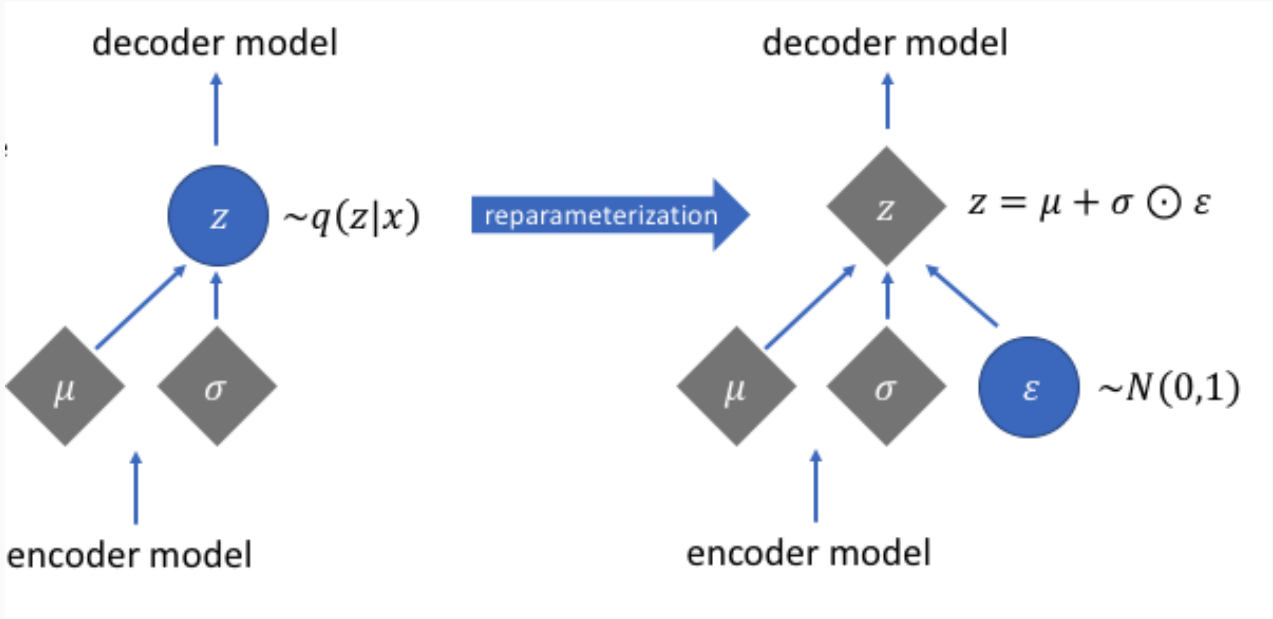
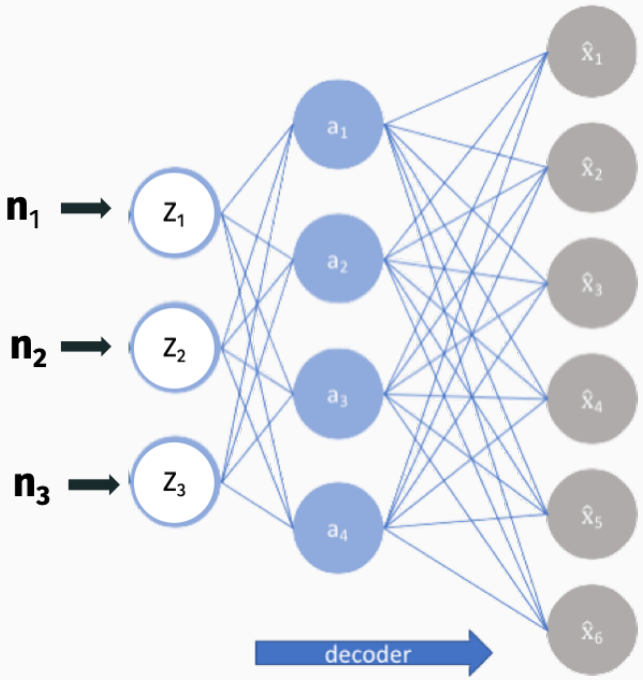
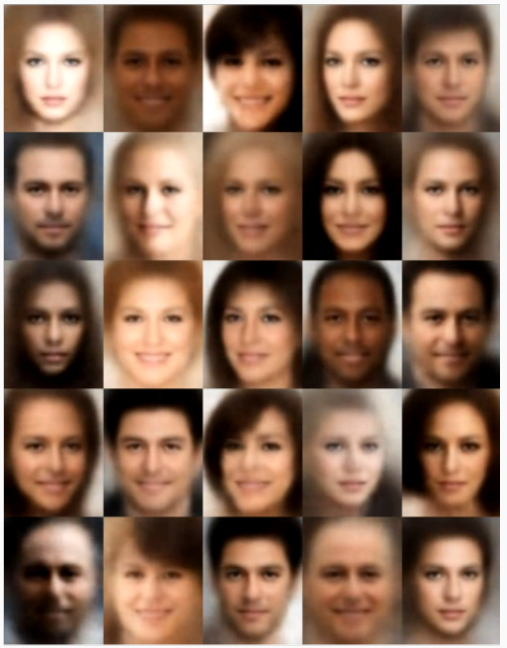


Image from `https://www.jeremyjordan.me/variational-autoencoders/`

Generative Adversarial Networks (GANs)

VAEs give very good results, but tends to produce images with immediately recognizable flaws (e.g. soft edges, high-frequency artifacts).



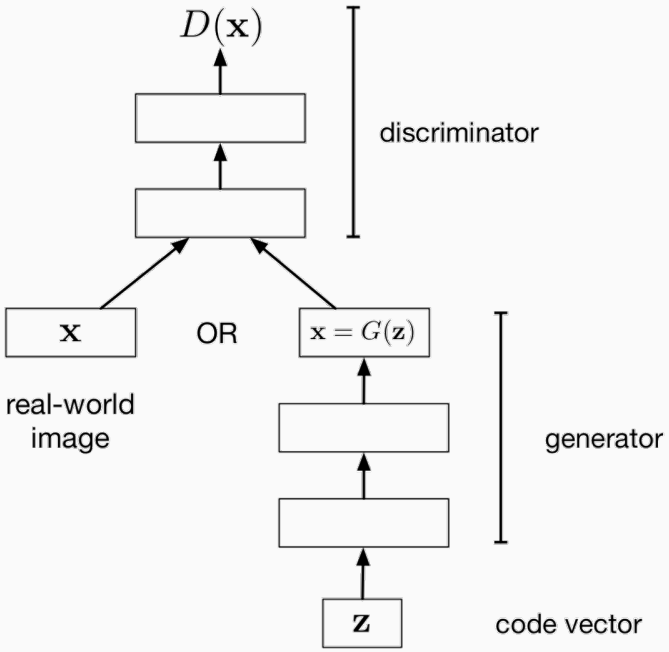


Generative Adversarial Networks (GANs)

Lots of efforts to hand-design regularizers that penalize images that don't look realistic to the human eye.

Main idea behind GANs: Use machine learning to automatically encourage realistic looking images.

Generative Adversarial Networks (GANs)

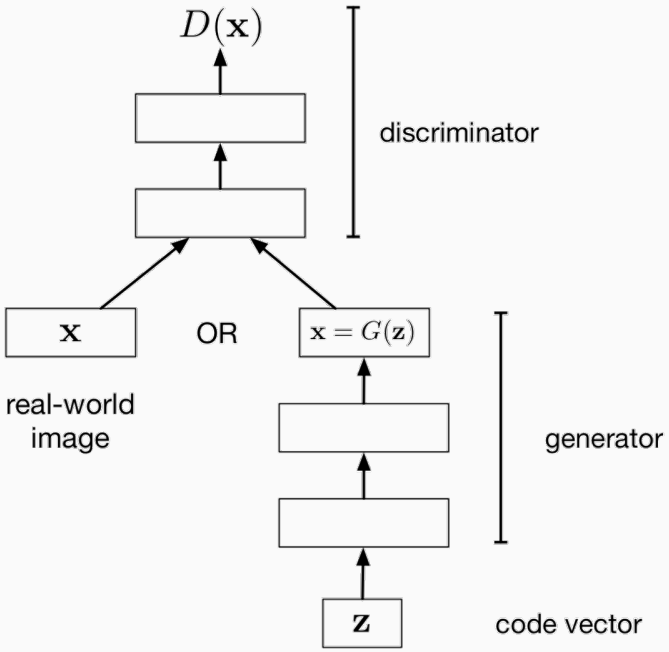


Let $\mathbf{x}_1, \ldots, \mathbf{x}_n$ be real images and let $\mathbf{z}_1, \ldots, \mathbf{z}_m$ be random code vectors. The goal of the discriminator is to output a number between $[0, 1]$ which is close to 0 if the image is fake, close to 1 if it's real.

Train weights of discriminator $D_{\boldsymbol{\theta}}$ to minimize:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log\left(D_{\boldsymbol{\theta}}(\mathbf{x}_i)\right) + \sum_{i=1}^{m} -\log\left(1 - D_{\boldsymbol{\theta}}(G_{\boldsymbol{\theta}'}(\mathbf{z}_i))\right)$$

Generative Adversarial Networks (GANs)



Goal of the generator $G_{\theta'}$ is the opposite. We want to maximize:

$$\max_{\theta'} \sum_{i=1}^{m} -\log\left(1 - D_{\theta}(G_{\theta'}(\mathbf{z}_i))\right)$$

This is called an "adversarial loss function". D is playing the role of the adversary.

Generative Adversarial Networks (GANs)

$$\boldsymbol{\theta}^*, \boldsymbol{\theta}'^* \text{ solve } \min_{\boldsymbol{\theta}} \max_{\boldsymbol{\theta}'} \sum_{i=1}^{n} -\log\left(D_{\boldsymbol{\theta}}(\mathbf{x}_i)\right) + \sum_{i=1}^{m} -\log\left(1 - D_{\boldsymbol{\theta}}(G_{\boldsymbol{\theta}'}(\mathbf{z}_i))\right)$$

This is called a minimax optimization problem. Really tricky to solve in practice.

- **Repeatedly play:** Fix one of $\boldsymbol{\theta}^*$ or $\boldsymbol{\theta}'^*$, train the other to convergence, repeat.
- **Simultaneous gradient descent:** Run a single gradient descent step for each of $\boldsymbol{\theta}^*, \boldsymbol{\theta}'^*$ and update D and G accordingly. Difficult to balance learning rates.
- Lots of tricks (e.g. slight different loss functions) can help.

Generative Adversarial Networks (GANs)

State of the art until a few years ago.

Quality of generative model

How to evaluate the quality of our model e.g., GAN? What do we expect from it?

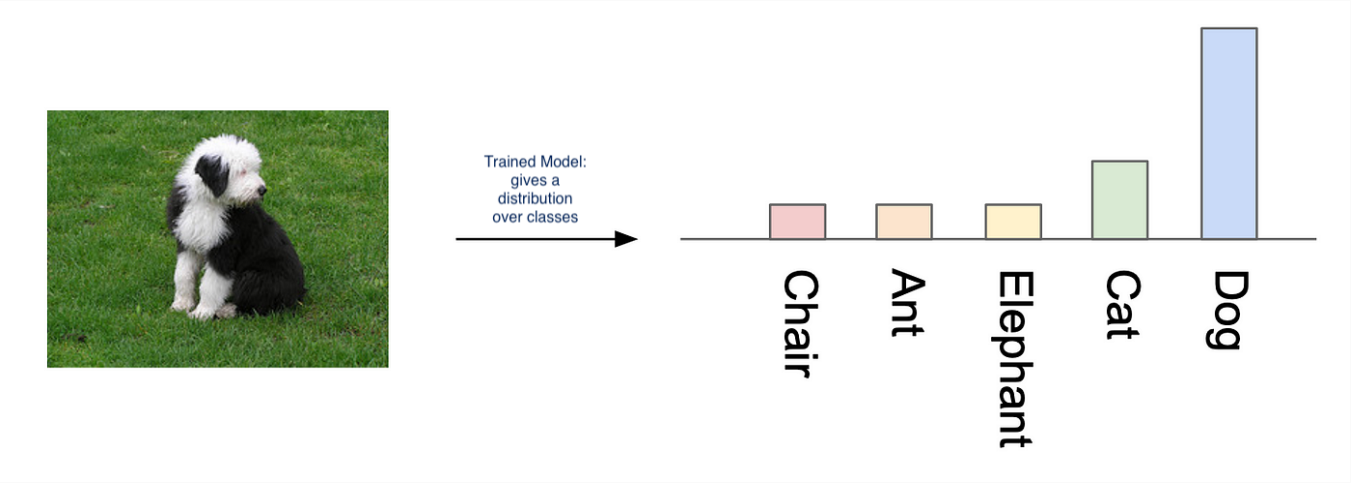
Quality of generative model

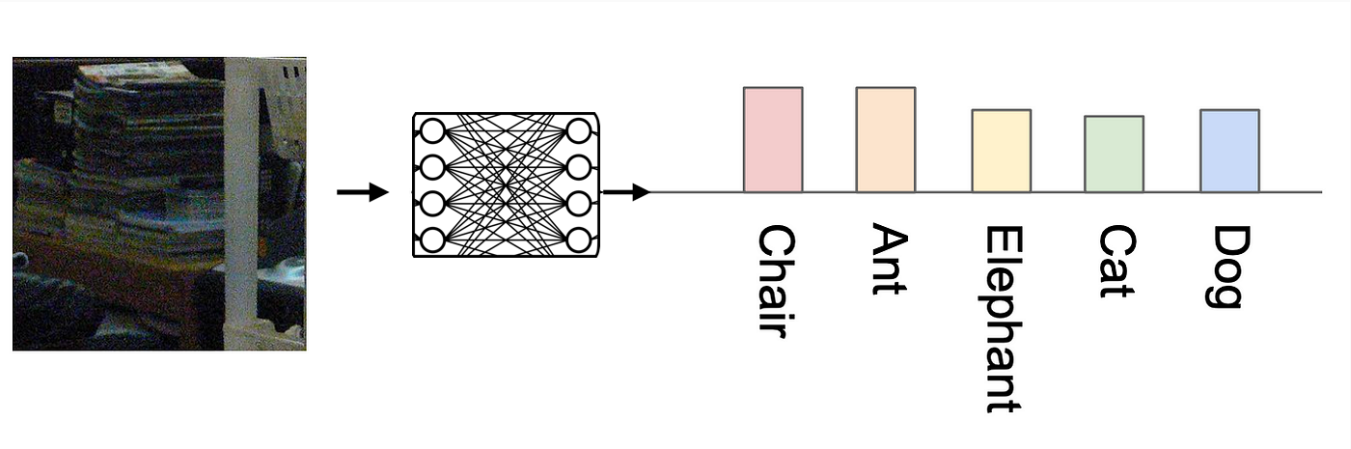
How to evaluate the quality of our model e.g., GAN? What do we expect from it?

- The images generated by our model should have variety (e.g., each image is a different breed of dog)
- Each image distinctly looks like something (e.g., one image is clearly a Poodle, the next a great example of a French Bulldog)
- ...

The Inception Score (IS)

Each image distinctly looks like something (e.g., one image is clearly a Poodle, the next a great example of a French Bulldog)

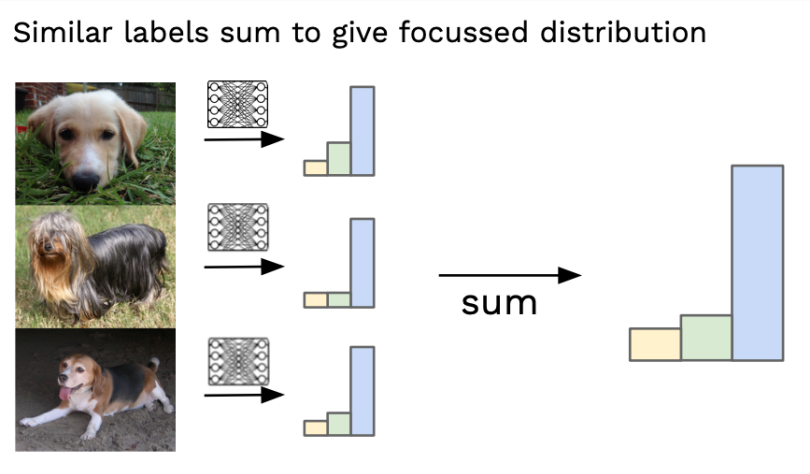


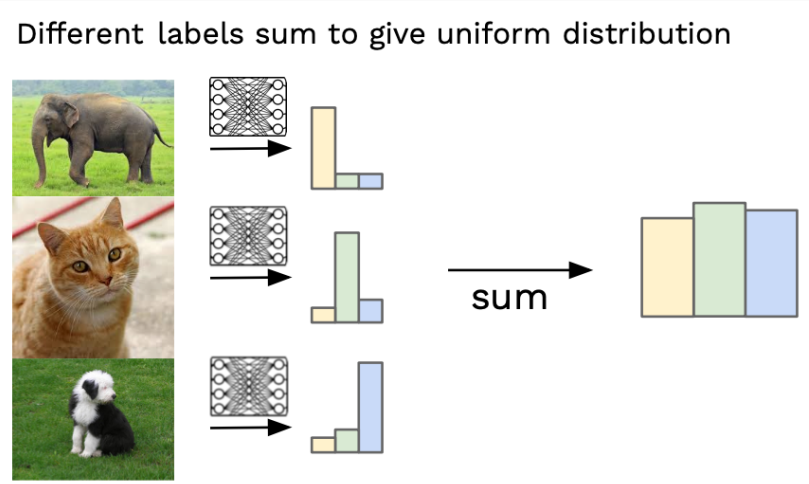


The Inception Score (IS)

The images generated by our model should have variety:

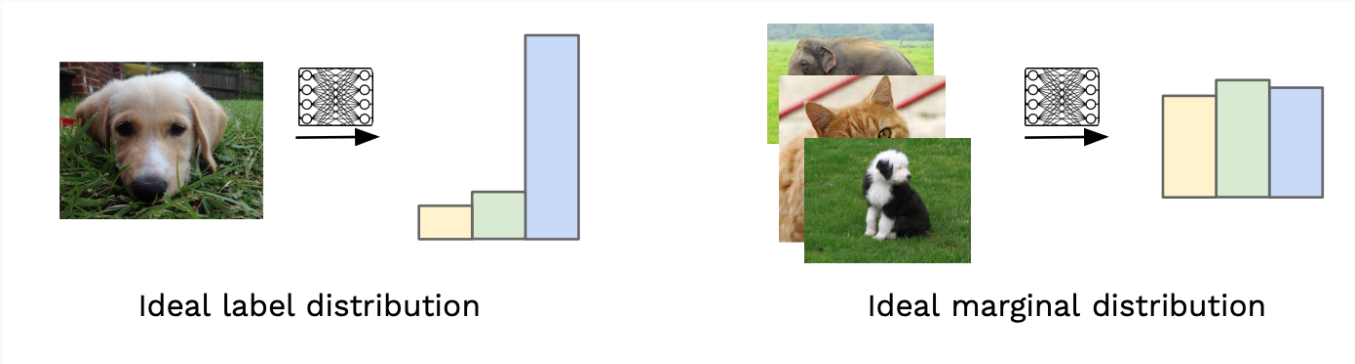
Generate a lot of images (50,000) using the model and sum their distributions.





The Inception Score (IS)

Higher KL divergence, means better score.



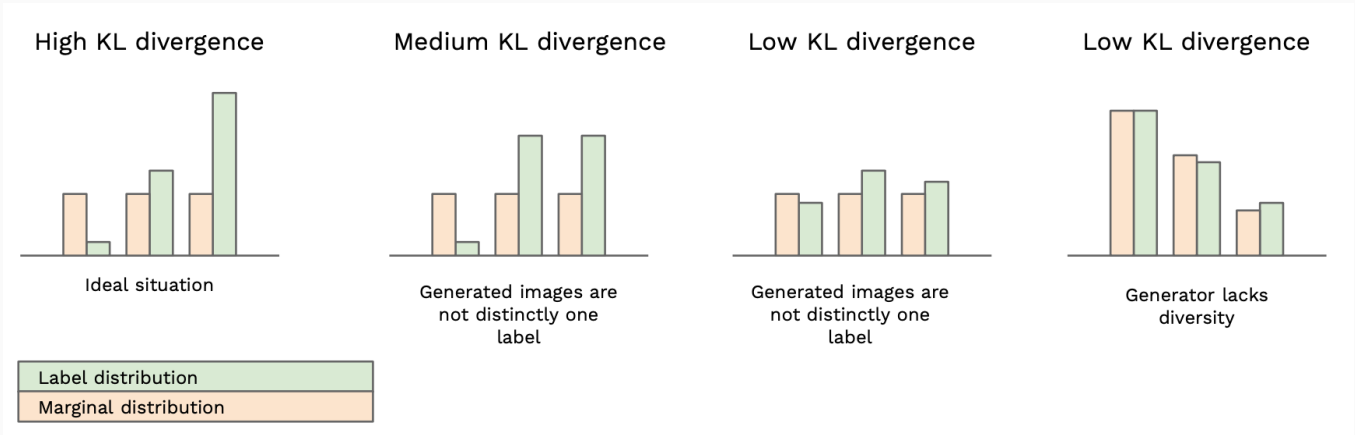


Image from
https://medium.com/octavian-ai/a-simple-explanation-of-the-inception-score-372dff6a8c7a

Diffusion Model

Diffusion

Auto-encoder/VAE, GAN approach: Input noise, map directly to image and vice versa.

Diffusion: Slowly move from noise to image and vice versa.

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

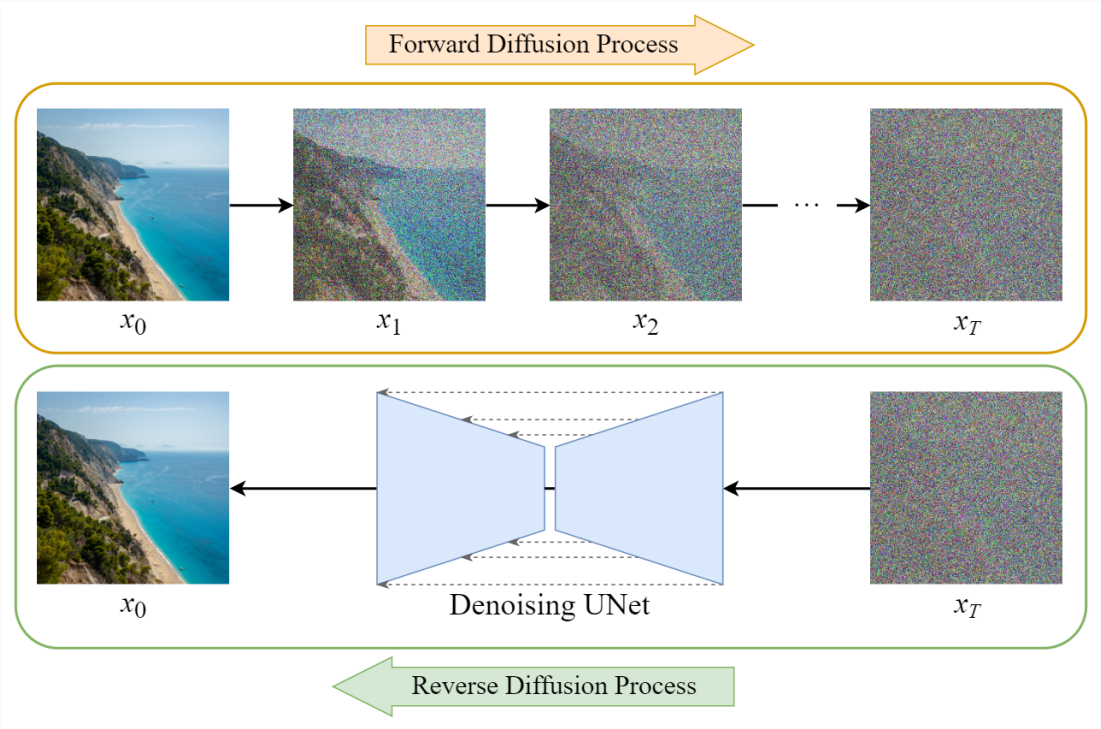
Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic

How diffusion models work

- Forward Process:
  - Gradually add noise to data until it becomes pure noise.
- Reverse Process:
  - Train a neural network to remove the noise step by step.

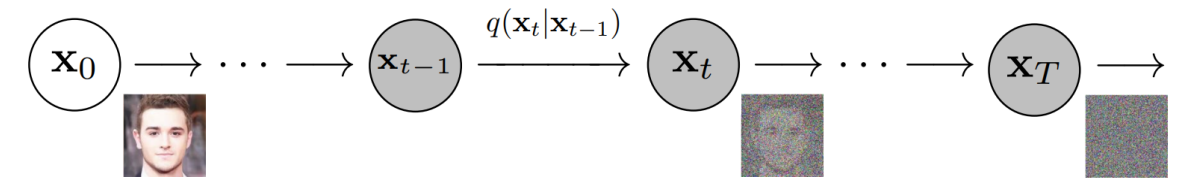


Key Question: How do we predict and reverse noise effectively?

Mathematical Formulation (1/2)

Forward Process (Adding Noise):

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$



- β_t: Noise schedule.
- After T steps, for large enough T, $\mathbf{x}_T$ is pure noise.

Cumulative Noise:

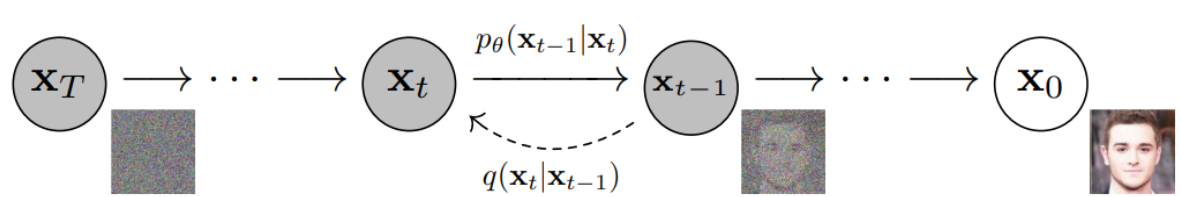
$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

with retention factor $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$.

Mathematical formulation (2/2)

Reverse Process (Denoising):

$$p_\theta(\mathbf{x}_{t-1}|x_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t))$$



- μ_θ: Predicted mean of the clean image.
- Σ_θ: Predicted variance (optional).

Training objective:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2\right]$$

Diffusion models vs VAEs

- Recall the goal of VAE was to have a probabilistic representation of latent space attributes. This was done in one shot, from image to Gaussian distributions.
- VAE decoder does the reverse in one shot. Takes Gaussian noise and returns an image.
- Diffusion model doing more or less the same but with many careful intermediate noising and denoising steps.
- There are fundamental differences ...

Diffusion process

- A diffusion process is a stochastic Markov process having continuous path
- stochastic Markov process: future state will only depend on the current state, knowing the past does not change anything
- continuous: no jumps
- Allows to transition from complex distributions to simple distributions

Forward process: a closer look

Forward process: $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$

Why does it converge to a simple distribution ?

Suppose the process is $\mathbf{x}_t = \alpha\mathbf{x}_{t-1} + \beta\mathcal{N}(0, I)$.

What values of α and β makes sense for the process?

- $\alpha = 0, \beta = 1$?
- $\alpha \geq 1, \beta \leq 1$?
- Seems something like $\alpha = \sqrt{0.99}, \beta = \sqrt{0.01}$ is appropriate.

Forward process: a closer look

Let's check if the following α and β converges:

$$\mathbf{x}_t = \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I)$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I) \\
&= \sqrt{1-\beta}(\sqrt{1-\beta}\mathbf{x}_{t-2} + \sqrt{\beta}\mathcal{N}(0, I)) + \sqrt{\beta}\mathcal{N}(0, I) \\
&= (\sqrt{1-\beta})^2\mathbf{x}_{t-2} + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{\beta}\mathcal{N}(0, I)
\end{aligned}$$

Forward process: a closer look

Let's check if the following α and β converges:

$$\mathbf{x}_t = \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I)$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I) \\
&= \sqrt{1-\beta}(\sqrt{1-\beta}\mathbf{x}_{t-2} + \sqrt{\beta}\mathcal{N}(0, I)) + \sqrt{\beta}\mathcal{N}(0, I) \\
&= (\sqrt{1-\beta})^2\mathbf{x}_{t-2} + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{\beta}\mathcal{N}(0, I) \\
&\cdots \\
&= (\sqrt{1-\beta})^t\mathbf{x}_{t-t} + \cdots + (\sqrt{1-\beta})^2\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) \\
&+ \sqrt{\beta}\mathcal{N}(0, I)
\end{aligned}$$

Forward process: a closer look

$$\mathbf{x}_t = (\sqrt{1-\beta})^t \mathbf{x}_0 + \cdots + (\sqrt{1-\beta})^2 \sqrt{\beta}\mathcal{N}(0, I) + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{\beta}\mathcal{N}(0, I)$$

- for large t, $(\sqrt{1-\beta})^t$ approaches to 0
- each of $(\sqrt{1-\beta})^i \sqrt{\beta}\mathcal{N}(0, I)$ is a Gaussian with mean 0 and variance $(1-\beta)^i \beta$.
- All these Gaussian can be written as one Gaussian distribution with variance:

$$\sum_{i=0}^{t} (1-\beta)^i \beta = \beta \frac{1-(1-\beta)^t}{1-(1-\beta)} \sim \frac{\beta}{\beta} = 1$$

For large enough t we converge to the Normal distribution $\mathcal{N}(0, I)$.

Forward process: a closer look

- In practice we do not use a fixed noise β
- Linear schedule noise: $\beta_1 = 0.0001$ and $\beta_T = 0.02$
- The number of steps is about 10000 (application dependent), but this is slow!
- Recall $\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I)$.
- Let's define $\alpha_t = 1 - \beta_t$ and do the recursive expansion.

Forward process: a closer look

$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\mathcal{N}(0, I)) + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= (\sqrt{\alpha_t\alpha_{t-1}})\mathbf{x}_{t-2} + \sqrt{1-\alpha_t+\alpha_t-\alpha_t\alpha_{t-1}}\sqrt{\beta}\mathcal{N}(0, I) \\
&\dots
\end{aligned}
$$

Forward process: a closer look

$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\mathcal{N}(0, I)) + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= (\sqrt{\alpha_t\alpha_{t-1}})\mathbf{x}_{t-2} + \sqrt{1-\alpha_t+\alpha_t-\alpha_t\alpha_{t-1}}\sqrt{\beta}\mathcal{N}(0, I) \\
&\cdots \\
&= \sqrt{\alpha_t\alpha_{t-1}\cdots\alpha_2\alpha_1}\mathbf{x}_0 + \sqrt{1-\alpha_t\alpha_{t-1}\cdots\alpha_2\alpha_1}\mathcal{N}(0, I) \\
&= \sqrt{\prod_{i=1}^{t}\alpha_i}\mathbf{x}_0 + \sqrt{1-\prod_{i=1}^{t}\alpha_i}\mathcal{N}(0, I) \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\mathcal{N}(0, I) \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon
\end{aligned}
$$

Reverse process: a closer look

Computing the reverse path (reverse denoising distribution) is not easy, but we know it is diffusion process.

We train a model to approximate the reverse distribution.

Similar to VAEs, the goal is to maximize a lower bound for the likelihood:

$$\log p(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \right]$$

A lot of effort goes into simplify this lower bound and exploiting the fact that we are dealing with diffusion process.

Reverse process: a closer look

At the end of the day, the model should learn the noise added

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2\right]$$

What does it mean? How do we actually do training ?

Reverse process: training

- Sample an image $\mathbf{x}_0$ and a time step t
- Sample a random noise ε.
- Get the noise image at time t: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$
- Feed $\mathbf{x}_t$ to the neural network.
- The model's predicted noise ε_θ should be close to ε.

Algorithm 1 Training

1: **repeat**
2: $\quad \mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $\quad t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: $\quad$ Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Reverse process: image generation

- Start from a noise image $\mathbf{x}_T \sim \mathcal{N}(0, I)$.
- Feed this to the trained neural network to predict a noise $\varepsilon_\theta(\mathbf{x}_T)$
- Given this predicted noise we can do denoising and obtain $\mathbf{x}_{T-1}$ (we skipped through the math here)
- Repeat the above until we get to $\mathbf{x}_0$.

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Semantic embeddings + diffusion

Text to image synthesis: Dall-E, Imagen, Stable Diffusion

"A chair that looks like a pineapple"

Diffusion

A demo for generating digits by training on MNIST.